PROJE ANALİZ RAPORU

Ders: BİL 204 / YAZILIM MÜHENDİSLİĞİ
Bölüm/Şube: BİLGİSAYAR MÜHENDİSLİĞİ / 2.SINIF
Grup Adı: CodeSentinel
Proje Adı: Statik Kod Analizi ve Hata Tespit Sistemi
Teslim Tarihi: 23.03.2026

Grup Üyeleri:
- Ferhat Enes KOCABIYIK (24100011813)
- Hacı Ömer OĞUZ (24100011082)
- Hüseyin Mert KANLIDERE (24100011021)
- Kemal BABAOĞLAN (25100011472)
- Mehmet Enes İYIŞENYÜREK (23100011005)
- Mehmet UÇAR (24100011044)

İçindekiler

İçindekiler 2
1. Giriş 3
2. Genel Bakış 3
3. Önerilen Sistem 5
3.1 İşlevsel Gereksinimler (Functional Requirements) 5
3.2 İşlevsel Olmayan Gereksinimler (Non-Functional Requirements) 6
3.3 Sözde Gereksinimler (Pseudo Requirements) 6
4 Sistem Modelleri 7
4.1 Kullanım Durumu (Use Case) Modeli 7
4.2 Sınıf Diyagramı 23
4.3 Durum Diyagramı 32
4.4 Aktivite Diyagramı 38
4.5 Sıra (Sequence) Diyagramı 44
4.6 Kullanıcı Arayüzü 48
5. Referanslar 50

1. Giriş

Günümüz yazılım dünyasında C programlama dili, yüksek performans ve donanım seviyesine yakın kontrol sağlaması nedeniyle yaygın olarak kullanılmaktadır. Ancak bu esnek ve düşük seviyeli yapı, geliştiricilerin hata yapma olasılığını artırmaktadır. Bu hataların önemli bir kısmı derleme aşamasında tespit edilememekte ve yalnızca çalışma zamanında ortaya çıkmaktadır.

Çalışma zamanında ortaya çıkan bu tür hatalar, özellikle güvenlik açıkları, bellek yönetimi problemleri ve sistem kararsızlıkları gibi kritik sorunlara yol açabilmektedir.

Bu proje kapsamında, C dilinde yazılmış kaynak kodları çalıştırmadan analiz edebilen bir statik kod analiz sistemi geliştirilmesi amaçlanmaktadır. Sistem, potansiyel hataları erken aşamada tespit ederek yazılım geliştirme sürecinde güvenilirliği artırmayı hedeflemektedir

2. Genel Bakış

C Statik Analiz Aracı, C dilinde yazılmış kaynak kodları analiz eden ve kullanıcıya hata, uyarı, metrik ve rapor çıktıları sunan bir masaüstü statik analiz aracıdır.

Sistem, C++ programlama dili kullanılarak ve nesne yönelimli programlama prensiplerine uygun şekilde geliştirilecektir. Araç; dosya yükleme, dosya doğrulama, yorum ayıklama, kod ayrıştırma, syntax hata tespiti, kural analizi, metrik hesaplama, sonuç gösterimi ve raporlama adımlarından oluşan bütünleşik bir analiz akışı sunacaktır.

C Statik Analiz Aracı, C dilinde yazılmış kaynak kodları analiz eden ve kullanıcıya hata, uyarı, metrik ve rapor çıktıları sunan bir masaüstü statik analiz aracıdır.

Sistem, C++ programlama dili kullanılarak ve nesne yönelimli programlama prensiplerine uygun şekilde geliştirilecektir. Araç; dosya yükleme, dosya doğrulama, yorum ayıklama, kod ayrıştırma, syntax hata tespiti, kural analizi, metrik hesaplama, sonuç gösterimi ve raporlama adımlarından oluşan bütünleşik bir analiz akışı sunacaktır.

Hedef Kullanıcılar

- Yazılım geliştiriciler: C dili ile sistem, gömülü veya düşük seviyeli yazılım geliştiren kullanıcılar.
- Sistem programcıları: Kod kalitesi ve hata tespiti konusunda hızlı geri bildirim almak isteyen kullanıcılar.
- Bilgisayar mühendisliği öğrencileri: Syntax, kod kalitesi ve statik analiz mantığını eğitim amaçlı inceleyen kullanıcılar.

Temel İşlevler

- C kaynak kodu dosyasını yükleme ve doğrulama.
- Yorum satırlarını ayıklama ve satır numarası bütünlüğünü koruma.
- Kodun tokenlara ayrılması ve AST oluşturulması.
- Syntax hatalarının tespiti ve satır bazında raporlanması.
- Kod kalitesi kurallarının çalıştırılması ve hata listesinin oluşturulması.
- Kod metriklerinin hesaplanması, sonuçların görüntülenmesi ve raporlanması.
- Bulgu satırına gitme, filtreleme ve ilgili kod parçasını vurgulama.

Kapsam ve Sınırlar

- Sistem yalnızca .c uzantılı C kaynak kodlarını analiz edecektir.
- Analiz işlemi tamamen statik analiz ile yapılacak, kod çalıştırılmayacaktır.
- Temel syntax ve kod kalitesi analizi kapsama dahil olacaktır; tam ölçekli derleyici/IDE entegrasyonu kapsam dışıdır.
- Sistem masaüstü ortamda çalışacak şekilde tasarlanacaktır.
- İleri düzey otomatik düzeltme mekanizmaları yerine hata tespiti, raporlama ve kullanıcıya yol gösterme işlevlerine odaklanılacaktır.

3. Önerilen Sistem

Önerilen sistem, C++ programlama dili kullanılarak geliştirilen ve C dilinde yazılmış kaynak kodları analiz eden bir statik kod analiz aracıdır. Sistem, nesne yönelimli programlama prensiplerine uygun olarak modüler bir mimari ile tasarlanacaktır.

Sistem; dosya girişinden analiz sonucunun ekranda gösterimine ve rapor olarak dışa aktarılmasına kadar uçtan uca bir akış sunacaktır. Use case modelinde tanımlanan işlevler, analiz sürecinin kullanıcı ve sistem etkileşimini açık biçimde ortaya koymaktadır.

3.1 İşlevsel Gereksinimler (Functional Requirements)

3.1.1 Dosya Alma ve Ön Doğrulama

- Sistem, kullanıcıdan .c uzantılı kaynak kod dosyasını almalıdır.
- Yüklenen dosyanın uzantısı, boş olup olmadığı ve okunabilirliği kontrol edilmelidir.
- Geçersiz dosya durumunda kullanıcıya uygun hata mesajı verilmelidir.

3.1.2 Ön İşleme ve Yorum Ayıklama

- Sistem, tek satırlı veya çok satırlı yorumları analiz dışı bırakmalıdır.
- Yorum ayıklama işlemi yapılırken satır numarası bütünlüğü korunmalıdır.
- String literal içindeki yorum benzeri karakter dizileri yanlış yorumlanmamalıdır.

3.1.3 Kod Ayrıştırma ve AST Oluşturma

- Sistem, kaynak kodu tokenlara ayırmalı ve bu tokenlardan AST üretmelidir.
- AST; fonksiyonlar, koşullar ve döngüler gibi yapıları temsil etmelidir.
- Ciddi syntax problemi oluşsa bile mümkün olduğunda kısmi ayrıştırma bilgisi tutulmalıdır.

3.1.4 Syntax Hatası Tespiti

- Sistem, eksik noktalı virgül, eşleşmeyen parantez ve benzeri syntax hatalarını tespit etmelidir.
- Her syntax hatası için satır numarası ve açıklayıcı hata mesajı üretilmelidir.

3.1.5 Kural Analizi ve Metrik Hesaplama

- Sistem, kullanılmayan değişken, başlatılmamış kullanım, magic number ve benzeri kuralları denetlemelidir.
- Sistem; LOC, fonksiyon sayısı, struct sayısı ve benzeri metrikleri hesaplamalıdır.
- Bulgu yoksa sistem bunu kullanıcıya açık biçimde bildirmelidir.

3.1.6 Sonuç Sunumu ve Raporlama

- Sistem, syntax hatalarını, kural ihlallerini ve metrik özetini arayüzde göstermelidir.
- Her bulgu için önem seviyesi atanmalı ve renk kodu ile gösterilmelidir.
- Kullanıcı analiz sonucunu raporlayabilmeli ve farklı dosya formatlarında kaydedebilmelidir.
- Kullanıcı seçtiği bulgu için ilgili satıra gidebilmeli, filtreleme yapabilmeli ve ilgili kod parçasını vurgulayabilmelidir.

3.2 İşlevsel Olmayan Gereksinimler (Non-Functional Requirements)

- **Performans**: Sistem, orta büyüklükteki bir C kaynak kodunu kabul edilebilir sürede analiz edebilmelidir.
- **Güvenilirlik**: Analiz sırasında oluşan hatalarda sistem tamamen durmamalı, mümkünse kısmi sonuç üretebilmelidir.
- **Kullanılabilirlik**: Arayüz; hata listesi, filtreleme ve satıra gitme işlemlerini kullanıcı dostu biçimde sunmalıdır.
- **Doğruluk**: Yanlış sonuç verme oranlarını azaltacak şekilde kurallar dikkatli tasarlanmalıdır.
- **Bakım Kolaylığı**: Analiz modülleri birbirinden bağımsız geliştirilebilir ve genişletilebilir yapıda olmalıdır.

3.3 Sözde Gereksinimler (Pseudo Requirements)

- Programlama dili olarak C++ kullanılacaktır.
- Yazılım geliştirme yaklaşımı nesne yönelimli programlama temelli olacaktır.
- Derleme ve proje yönetimi için CMake tercih edilecektir.

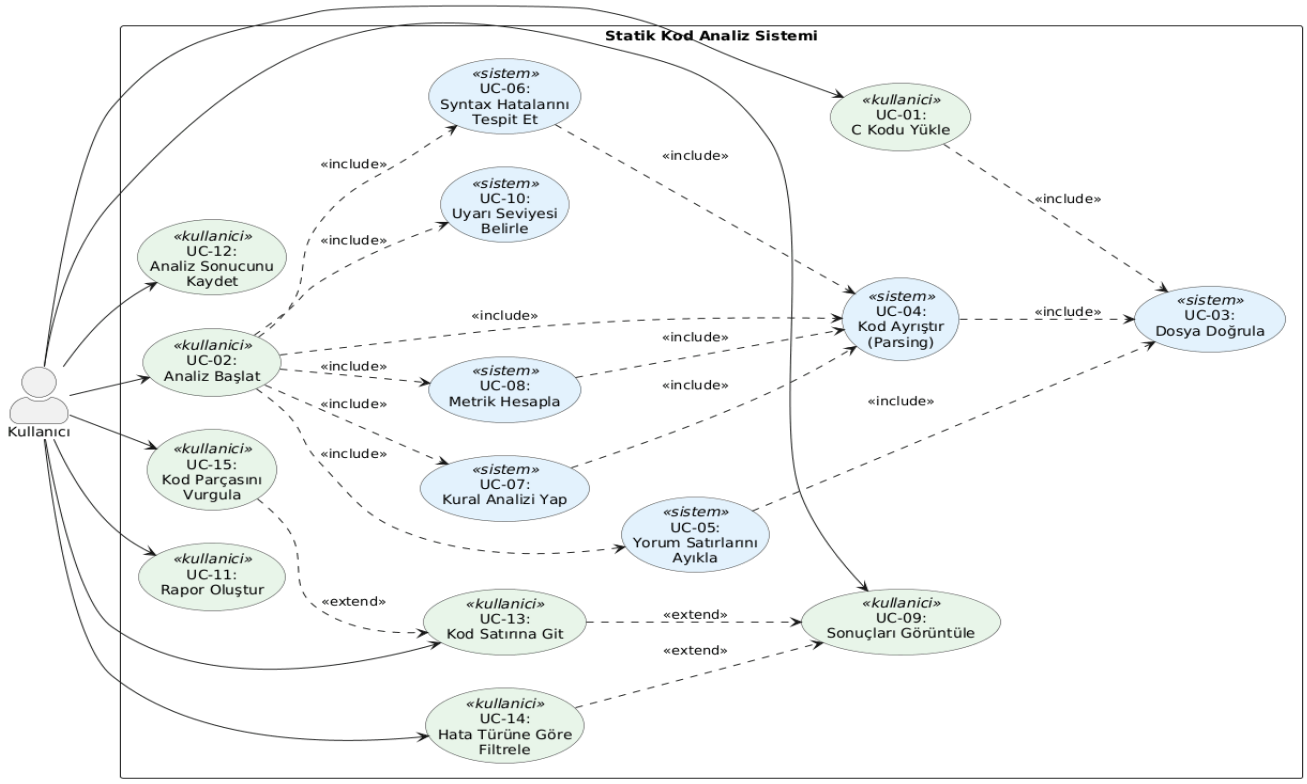
- Versiyon kontrol ve ekip çalışması için GitHub kullanılacaktır.
- Sistem masaüstü ortam için tasarlanacaktır ve modüler bir mimari benimseyecektir.

4. Sistem Modelleri

4.1 Kullanım Durumu (Use Case) Modeli

Bu bölümde sistemin tüm fonksiyonel gereksinimleri kullanım durumları (Use Case) ile modellenmiştir.
Sistem C dilinde yazılmış kaynak kodları analiz eden bir statik analiz aracıdır.

Tüm sistem tek bir Use Case diyagramı ile temsil edilecektir. Diyagramda kullanıcı ve sistem etkileşimi gösterilecektir. Her kullanım durumu aşağıda detaylı şekilde açıklanmıştır.



1. Bu diyagram, sistemin tüm fonksiyonel gereksinimlerini kullanıcı ve sistem etkileşimi perspektifinden modellemektedir. Diyagramda tek bir aktör olan Kullanıcı ile sistem arasındaki etkileşimler toplam 15 kullanım durumu (UC-01 – UC-15) ile gösterilmiştir.
2. Kullanıcı tarafından doğrudan başlatılan kullanım durumları (yeşil tonlar) şunlardır:
3. UC-01 (C Kodu Yükle): Kullanıcının .c uzantılı dosyayı sisteme yüklemesi.
4. UC-02 (Analiz Başlat): Analiz sürecinin tüm alt adımlarını tetikleyen orkestrasyon noktası.
5. UC-09 (Sonuçları Görüntüle): Analiz sonuçlarının kategorize edilerek listelenmesi.
6. UC-11 (Rapor Oluştur): Analiz sonuçlarının yapılandırılmış rapor olarak derlenmesi.
7. UC-12 (Analiz Sonucunu Kaydet): Sonuçların TXT, JSON veya HTML formatında diske kaydedilmesi.
8. UC-13 (Kod Satırına Git): Seçilen bulgudaki satıra otomatik navigasyon.
9. UC-14 (Hata Türüne Göre Filtrele): Sonuçların önem seviyesine göre filtrelenmesi.
10. UC-15 (Kod Parçasını Vurgula): İlgili kod bloğunun görsel olarak vurgulanması.
11. Sistem tarafından otomatik yürütülen kullanım durumları (mavi tonlar) <<include>> ilişkisi ile bağlanmıştır:
12. UC-03 (Dosya Doğrula): Uzantı, boşluk ve okunabilirlik kontrolü. UC-01 tarafından otomatik tetiklenir.
13. UC-04 (Kod Ayrıştır): Tokenization ve AST oluşturma. Sonraki tüm analizlerin temelini oluşturur.
14. UC-05 (Yorum Satırlarını Ayıkla): Tek ve çok satırlı yorumların analiz dışı bırakılması.
15. UC-06 (Syntax Hatalarını Tespit Et): Eksik noktalı virgül, eşleşmeyen parantez vb. hataların bulunması.
16. UC-07 (Kural Analizi Yap): Kullanılmayan değişken, magic number gibi kod kalitesi kurallarının denetlenmesi.
17. UC-08 (Metrik Hesapla): LOC, fonksiyon sayısı, struct sayısı gibi metriklerin hesaplanması.
18. UC-10 (Uyarı Seviyesi Belirle): Bulgulara Kritik (kırmızı), Uyarı (sarı) veya Bilgi (mavi) seviyesi atanması.

Not: Sistemde birincil aktör kullanıcıdır. Ancak bazı kullanım durumları kullanıcı tarafından doğrudan başlatılmayan, sistem tarafından otomatik yürütülen alt süreçlerdir. Bu nedenle ilgili kullanım durumlarında aktör olarak sistem belirtilmiştir.

[UC-> Use Case]

UC-01: C Kodu Yükle

Alan	Değer
Aktör	Kullanıcı
Giriş Koşulu	Sistem çalışır durumda olmalıdır
Çıkış Koşulu	Dosya başarıyla yüklenir veya hata mesajı döner

Ana Akış:

1. Kullanıcı, arayüzden "Dosya Aç" veya benzeri bir buton ile .c uzantılı dosya seçer.
2. Sistem dosyayı okur ve belleğe alır.
3. Dosya içeriği editör alanında kullanıcıya gösterilir.

Alternatif Akış:

- Kullanıcı .c uzantısı dışında bir dosya seçerse → sistem uyarı verir ve yükleme iptal edilir.
- Dosya boyutu çok büyükse → sistem bir sınır kontrolü yaparak kullanıcıyı bilgilendirir.
- Dosya boşsa → sistem "Boş dosya yüklendi" uyarısı verir.

Detay: Bu use case, analiz sürecinin ilk adımıdır. Dosya belleğe alındıktan sonra Dosya Doğrula (UC-03) use case'i otomatik olarak tetiklenir (<<include>> ilişkisi).

UC-02: Analiz Başlat

Alan	Değer
Aktör	Kullanıcı
Giriş Koşulu	Geçerli bir .c dosyası yüklenmiş olmalıdır
Çıkış Koşulu	Analiz süreci tamamlanır ve sonuçlar hazır hale gelir

Ana Akış:

1. Kullanıcı "Analiz Başlat" butonuna tıklar.
2. Sistem sırasıyla şu alt süreçleri otomatik olarak tetikler:
    - Yorum Satırlarını Ayıkla (UC-05)
    - Kod Ayrıştır (UC-04)
    - Syntax Hatalarını Tespit Et (UC-06)
    - Kural Analizi Yap (UC-07)
    - Metrik Hesapla (UC-08)
    - Uyarı Seviyesi Belirle (UC-10)
3. **Analiz tamamlandığında sonuçlar ekranda listelenir.**

Alternatif Akış:

- Dosya doğrulama başarısız olursa → analiz başlamaz, kullanıcıya hata mesajı gösterilir.

- Analiz sırasında beklenmedik bir hata oluşursa → sistem kısmi sonuçları gösterir ve hangi adımda hata oluştuğunu belirtir.

Detay: Bu use case, sistemin en önemli orkestrasyon noktasıdır. Birden fazla alt use case'i <<include>> ilişkisiyle sırasıyla çağırır.

UC-03: Dosya Doğrula

Alan	Değer
Aktör	Sistem (otomatik)
Giriş Koşulu	Bir dosya yüklenmiş olmalıdır
Çıkış Koşulu	Dosya geçerli veya geçersiz olarak işaretlenir

Ana Akış:

1. Sistem dosyanın uzantısının .c olup olmadığını kontrol eder.
2. Dosya içeriğinin metin formatında olup olmadığı doğrulanır (binary dosya kontrolü).
3. Dosyanın boş olup olmadığı kontrol edilir.
4. Doğrulama başarılı ise analiz sürecine devam edilir.

Alternatif Akış:

- Uzantı .c değilse → "Desteklenmeyen dosya formatı" hatası döner.
- Dosya boşsa → "Dosya içeriği bulunamadı" uyarısı verilir.
- Dosya bozuk/okunamaz ise → "Dosya okunamıyor" hatası döner.

Detay: Bu use case doğrudan kullanıcı tarafından başlatılmaz. Dosya yükleme sonrası sistem tarafından otomatik çağrılır. Doğrulama başarısız olursa analiz süreci hiç başlamaz.

UC-04: Kod Ayrıştır (Parsing)

Alan	Değer
Aktör	Sistem (otomatik)
Giriş Koşulu	Dosya doğrulanmış ve belleğe alınmış olmalıdır
Çıkış Koşulu	Kaynak kod token'lara ayrılır ve AST oluşturulur

Ana Akış:

1. Tokenization (Lexical Analysis): Kaynak kod, anlamlı birimlerine (token) ayrılır — anahtar kelimeler (int, if, return), operatörler (+, =), literaller, tanımlayıcılar vb.

2. AST (Abstract Syntax Tree) Oluşturma: Token'lar kullanılarak kodun yapısal ağacı oluşturulur. Bu ağaç, fonksiyon tanımları, döngüler, koşullu ifadeler gibi yapısal blokları temsil eder.

Alternatif Akış:

- Tokenization sırasında tanınmayan karakter bulunursa → hata listesine eklenir ve işleme devam edilir.
- AST oluşturulamazsa (ciddi syntax hatası) → kısmi ağaç oluşturulur ve hatalar raporlanır.

Detay: Bu adım, sonraki tüm analizlerin (syntax kontrolü, kural analizi, metrik hesaplama) temelini oluşturur. AST olmadan statik analiz yapılamaz.

UC-05: Yorum Satırlarını Ayıkla

Alan	Değer
Aktör	Sistem (otomatik)
Giriş Koşulu	Kaynak kod sisteme yüklenmiş olmalıdır
Çıkış Koşulu	Tek satırlı (//) ve çok satırlı (/* */) yorumlar analiz dışı bırakılır

Ana Akış:

1. Sistem, kodda tek satırlı yorumları (//...) tespit eder.
2. Çok satırlı yorumları (/* ... */) tespit eder.
3. Yorum içeriklerini ayıklar ve analiz kapsamından çıkarır.
4. Satır numarası bütünlüğü korunur (yorum çıkarılsa bile satır numaraları değişmez).

Alternatif Akış:

- String literal içindeki // veya /* karakterleri yorum olarak algılanmamalıdır (örn: char *s = "// bu yorum değil";).
- İç içe yorum varsa → uyarı verilir (C dilinde /* /* */ */ geçersizdir).

Detay: Yorumların ayıklanması, yanlış pozitif hata tespitini önlemek için kritiktir. Yorum içindeki kod benzeri ifadeler hata olarak raporlanmamalıdır.

UC-06: Syntax Hatalarını Tespit Et

Alan	Değer
Aktör	Sistem (otomatik)

Alan	Değer
Giriş Koşulu	Kod ayrıştırma (parsing) işlemi tamamlanmış olmalıdır
Çıkış Koşulu	Tüm syntax hataları tespit edilir ve hata listesi oluşturulur

Ana Akış:

1. Sistem, AST oluşturma sırasında karşılaşılan syntax hatalarını toplar.
2. Her hata için şu bilgiler kaydedilir:
    - Satır numarası
    - Hatanın türü (eksik noktalı virgül, eşleşmeyen parantez vb.)
    - Hata mesajı
3. Hata listesi oluşturulur.

Tespit Edilen Hata Örnekleri:

- Eksik ; (noktalı virgül)
- Eşleşmeyen {}, (), [] parantezleri
- Eksik veya fazla fonksiyon parametresi
- Geçersiz tip tanımlamaları

Alternatif Akış:

- Hiç syntax hatası bulunmazsa → boş hata listesi döner, analiz devam eder.
- Çok sayıda hata bulunursa → sistem bir üst sınır belirleyerek ilk N hatayı raporlayabilir.

UC-07: Kural Analizi Yap

Alan	Değer
Aktör	Sistem (otomatik)
Giriş Koşulu	Kod ayrıştırma tamamlanmış olmalıdır
Çıkış Koşulu	Kod kalitesi kurallar çerçevesinde değerlendirilir

Ana Akış:

1. Sistem, önceden tanımlı kod kalitesi kurallarını AST üzerinde çalıştırır.
2. Her kural ihlali için satır numarası, kural adı ve açıklama kaydedilir.
3. Sonuçlar bir ihlal listesi olarak oluşturulur.

Örnek Kurallar:

- Kullanılmayan değişken tespiti
- Başlatılmamış (uninitialized) değişken kullanımı
- Sonsuz döngü potansiyeli
- Magic number kullanımı (sabit yerine sayısal değer)
- Fonksiyon uzunluğu sınırı aşımı
- goto kullanımı uyarısı

Alternatif Akış:

- Hiçbir kural ihlali bulunmazsa → "Kod kalitesi kurallarına uygun" mesajı verilir.

Detay: Kural analizi, syntax doğruluğunun ötesinde kod kalitesi ve güvenlik açısından önemlidir.

UC-08: Metrik Hesapla

Alan	Değer
Aktör	Sistem (otomatik)
Giriş Koşulu	Kod ayrıştırma tamamlanmış olmalıdır
Çıkış Koşulu	Kod metrikleri hesaplanır

Ana Akış:

1. Sistem şu metrikleri hesaplar:
    - Toplam satır sayısı (LOC — Lines of Code)
    - Fonksiyon sayısı (tanımlanan fonksiyonların toplam adedi)
    - Struct sayısı (tanımlanan yapı/struct adedi)

Ek Hesaplanabilecek Metrikler:

- Boş satır sayısı
- Yorum satırı sayısı
- Ortalama fonksiyon uzunluğu
- Değişken sayısı

Detay: Metrikler, kodun karmaşıklığı ve bakım kolaylığı hakkında nicel bilgi sunar. Rapor oluşturma (UC-11) aşamasında bu metrikler rapora dahil edilir.

UC-09: Sonuçları Görüntüle

Alan	Değer
Aktör	Kullanıcı
Giriş Koşulu	Analiz tamamlanmış olmalıdır
Çıkış Koşulu	Sonuçlar arayüzde listelenir

Ana Akış:

1. Analiz bittiğinde sistem, sonuçları kategorize ederek arayüzde listeler:
    - Syntax hataları
    - Kural ihlalleri
    - Metrik özeti
2. Kullanıcı sonuçları inceler.
3. Her sonuç kaydı; satır numarası, mesaj ve önem seviyesi (renk kodu) ile gösterilir.

Alternatif Akış:

- Hiç sonuç yoksa → "Herhangi bir hata veya uyarı bulunamadı" mesajı gösterilir.

Detay: Bu arayüz, kullanıcının analiz sonuçlarıyla doğrudan etkileşime girebildiği ana ekrandır. Hata Türüne Göre Filtrele (UC-14) ve Kod Satırına Git (UC-13) gibi use case'ler bu ekrandan tetiklenebilir.

UC-10: Uyarı Seviyesi Belirle

Alan	Değer
Aktör	Sistem (otomatik)

Alan	Değer
Giriş Koşulu	Hatalar ve ihlaller tespit edilmiş olmalıdır
Çıkış Koşulu	Her bulguya bir önem seviyesi ve renk atanır

Ana Akış:

1. Sistem her bulgunun türünü değerlendirir.
2. Bulgunun ciddiyetine göre bir uyarı seviyesi atanır:
    - Kritik (Kırmızı): Derleme hatasına neden olacak sorunlar (eksik ;, eşleşmeyen parantez)
    - Uyarı (Sarı): Potansiyel sorunlar, kötü pratikler (kullanılmayan değişken, magic number)
    - Bilgi (Mavi): Bilgilendirme amaçlı öneriler (isimlendirme kuralları, stil önerileri)
3. Renk kodları arayüzde görsel olarak uygulanır.

Detay: Uyarı seviyeleri, kullanıcının önce kritik sorunlara odaklanmasını sağlar. Raporlama ve filtreleme işlemleri de bu seviye bilgisini kullanır.

UC-11: Rapor Oluştur

Alan	Değer
Aktör	Kullanıcı
Giriş Koşulu	Analiz tamamlanmış olmalıdır

Alan	Değer
Çıkış Koşulu	Analiz raporu oluşturulur

Ana Akış:

1. Kullanıcı "Rapor Oluştur" butonuna tıklar.
2. Sistem, analiz sonuçlarını yapılandırılmış bir formatta derler:
    - Dosya bilgisi (ad, boyut, tarih)
    - Metrik özeti (satır, fonksiyon, struct sayıları)
    - Hata ve uyarı listesi (önem seviyesine göre gruplandırılmış)
    - Genel değerlendirme
3. **Rapor kullanıcıya sunulur.**

Alternatif Akış:

- Analiz henüz tamamlanmamışsa → "Önce analiz çalıştırın" uyarısı gösterilir.

UC-12: Analiz Sonucunu Kaydet

Alan	Değer
Aktör	Kullanıcı
Giriş Koşulu	Analiz tamamlanmış olmalıdır
Çıkış Koşulu	Sonuçlar bir dosyaya kaydedilir

Ana Akış:

1. Kullanıcı "Kaydet" butonuna tıklar.
2. Sistem, kaydetme formatını ve konumunu belirler (veya kullanıcıdan ister).
3. Analiz sonuçları dosya olarak diske yazılır (örn: .txt, .json, .html).
4. Kayıt tamamlandığında kullanıcıya onay mesajı gösterilir.

Alternatif Akış:

- Dosya yazma yetkisi yoksa → "Yazma izni yok" hatası verilir.
- Disk alanı yetersizse → "Yetersiz disk alanı" uyarısı verilir.

UC-13: Kod Satırına Git

Alan	Değer
Aktör	Kullanıcı
Giriş Koşulu	Bir hata veya uyarı seçilmiş olmalıdır
Çıkış Koşulu	İlgili satır editör alanında gösterilir ve vurgulanır

Ana Akış:

1. Kullanıcı, sonuç listesindeki bir hataya/uyarıya tıklar.
2. Sistem, editör alanında ilgili satır numarasına otomatik olarak kaydırma yapar.
3. İlgili satır görsel olarak vurgulanır (highlight).

Detay: Bu özellik, kullanıcının hata ile kaynak kod arasında hızlıca geçiş yapmasını sağlayan bir navigasyon aracıdır. Büyük dosyalarda kullanıcı deneyimini önemli ölçüde iyileştirir.

UC-14: Hata Türüne Göre Filtrele

Alan	Değer
Aktör	Kullanıcı
Giriş Koşulu	Analiz sonuçları mevcut olmalıdır
Çıkış Koşulu	Sonuçlar seçilen filtreye göre listelenir

Ana Akış:

1. Kullanıcı, filtre seçeneklerinden bir veya daha fazla kategori seçer:
    - Sadece kritik hatalar
    - Sadece uyarılar
    - Sadece bilgi notları
    - Belirli kural türleri
2. Sistem, sonuç listesini filtreye göre günceller.
3. Filtre kaldırıldığında tüm sonuçlar tekrar gösterilir.

Detay: Çok sayıda bulgu olduğunda kullanıcının ilgili sonuçlara odaklanmasını sağlar. Uyarı seviyesine (UC-10) göre filtreleme en yaygın kullanım senaryosudur.

UC-15: Kod Parçasını Vurgula

Alan	Değer
Aktör	Kullanıcı

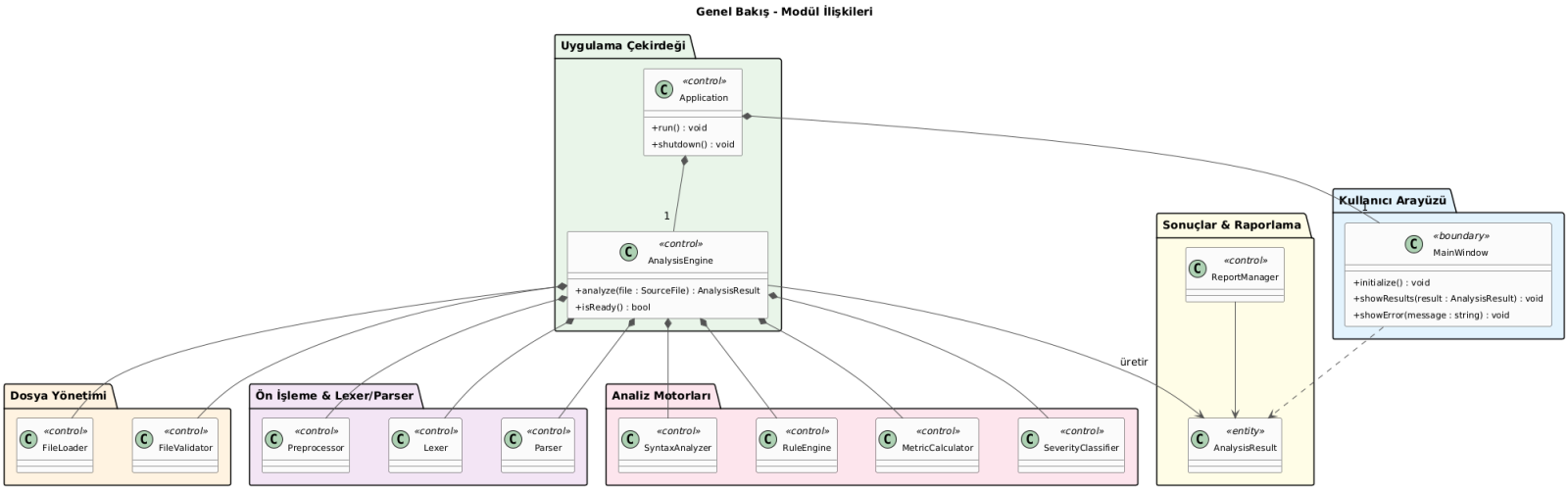
Alan	Değer
Giriş Koşulu	Bir hata veya uyarı seçilmiş olmalıdır
Çıkış Koşulu	İlgili kod bloğu görsel olarak vurgulanır

Ana Akış:

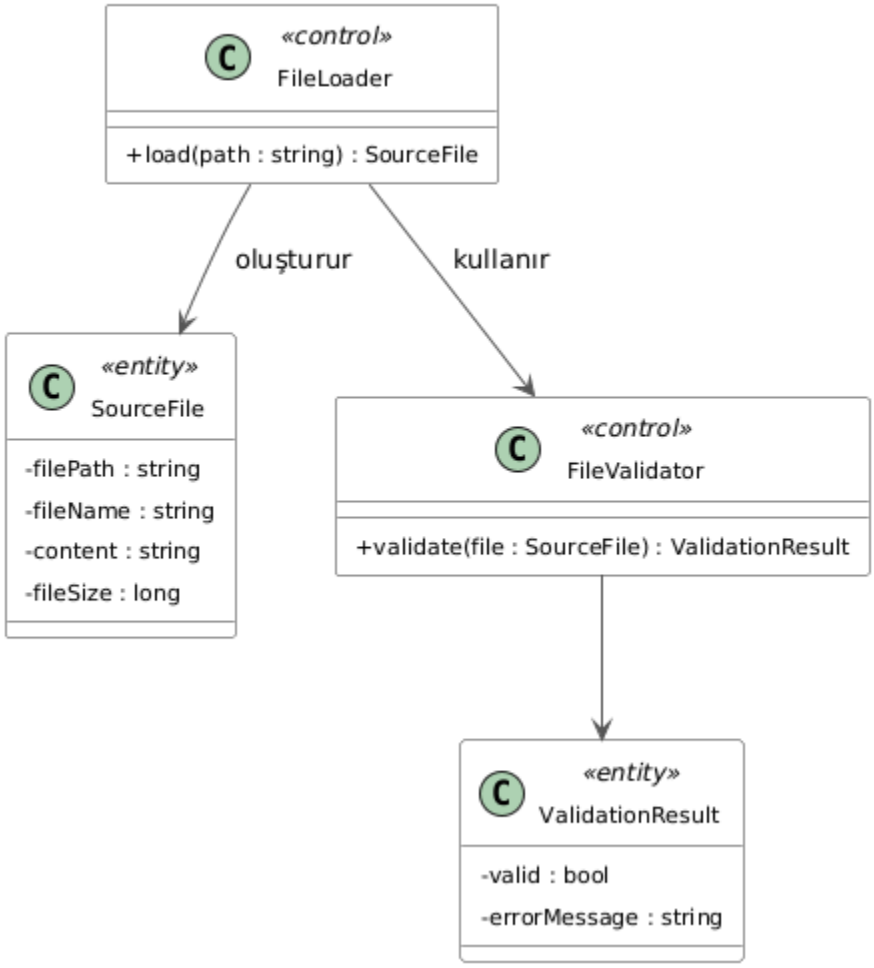
1. Kullanıcı, sonuç listesinden bir bulguyu seçer.
2. Sistem, hatanın/uyarının ilişkili olduğu kod parçasını (tek satır veya çok satırlık blok) belirler.
3. İlgili kod parçası, uyarı seviyesine uygun renkte (kırmızı/sarı/mavi) görsel olarak vurgulanır.

Detay: Kod Satırına Git (UC-13) ile birlikte çalışır. UC-13 satıra navigasyon sağlarken, UC-15 ilgili bölümü görsel olarak ön plana çıkarır. Birlikte kullanıldığında hata tespiti ve düzeltme süreci hızlanır.

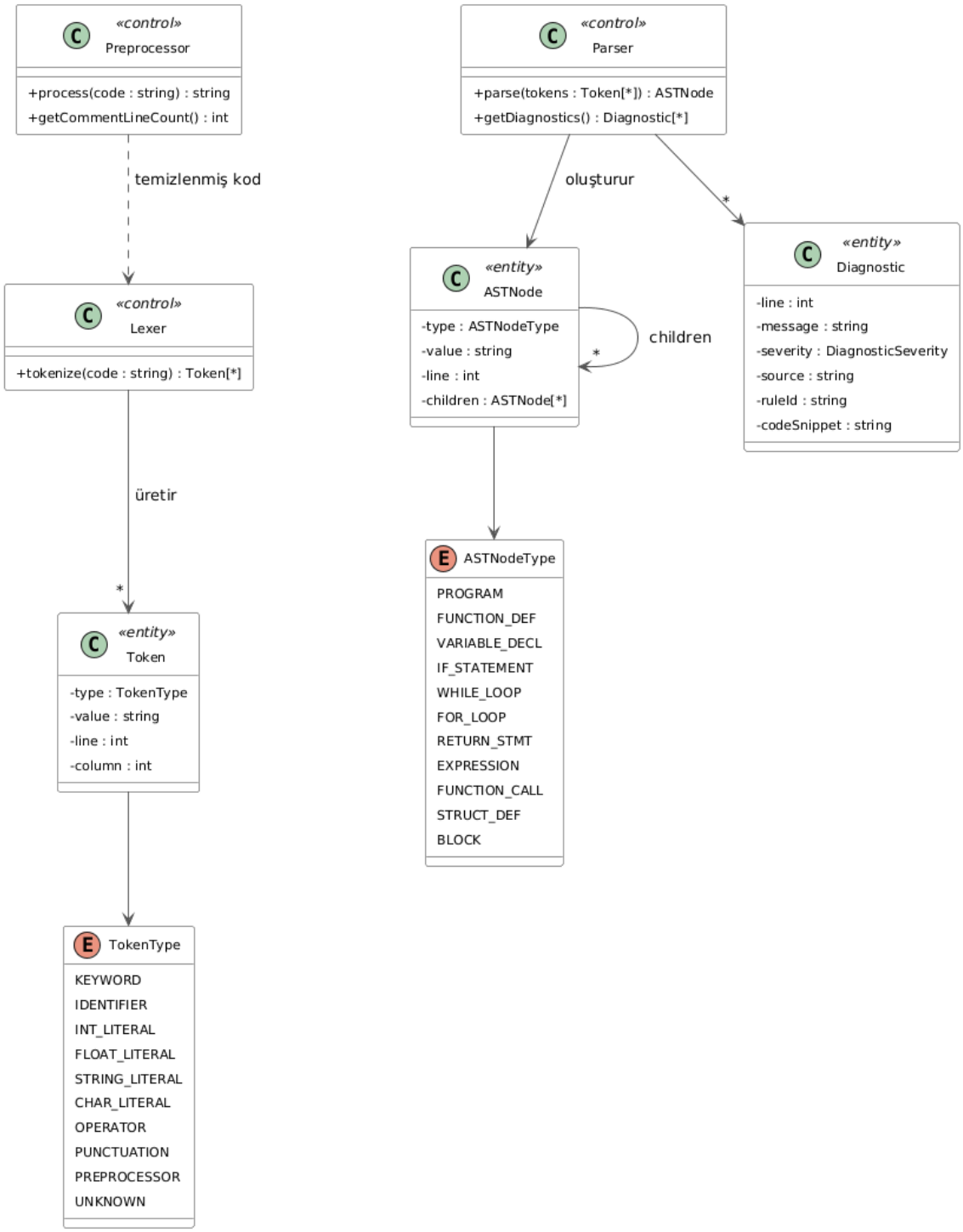
4.2 Class Diyagramı



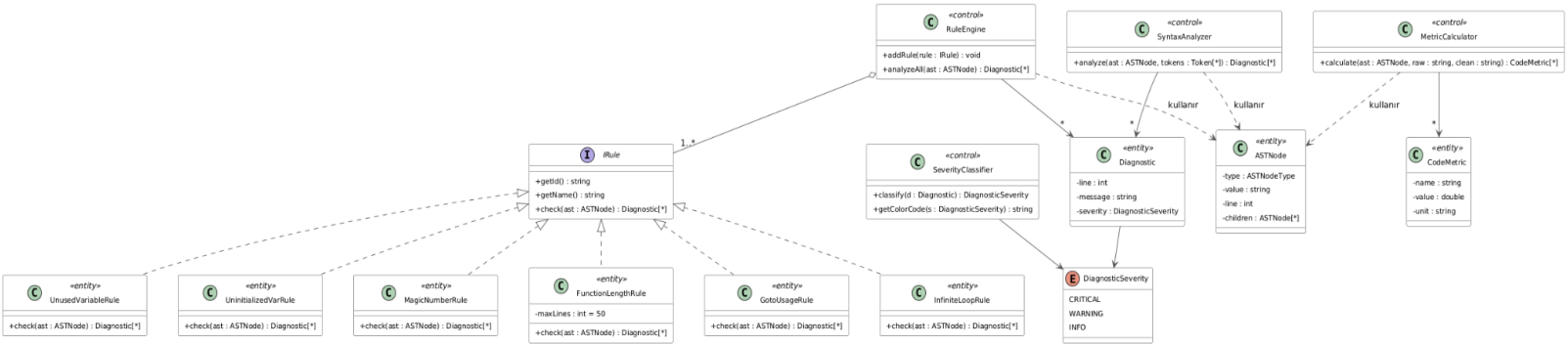
Dosya Yönetimi Modülü



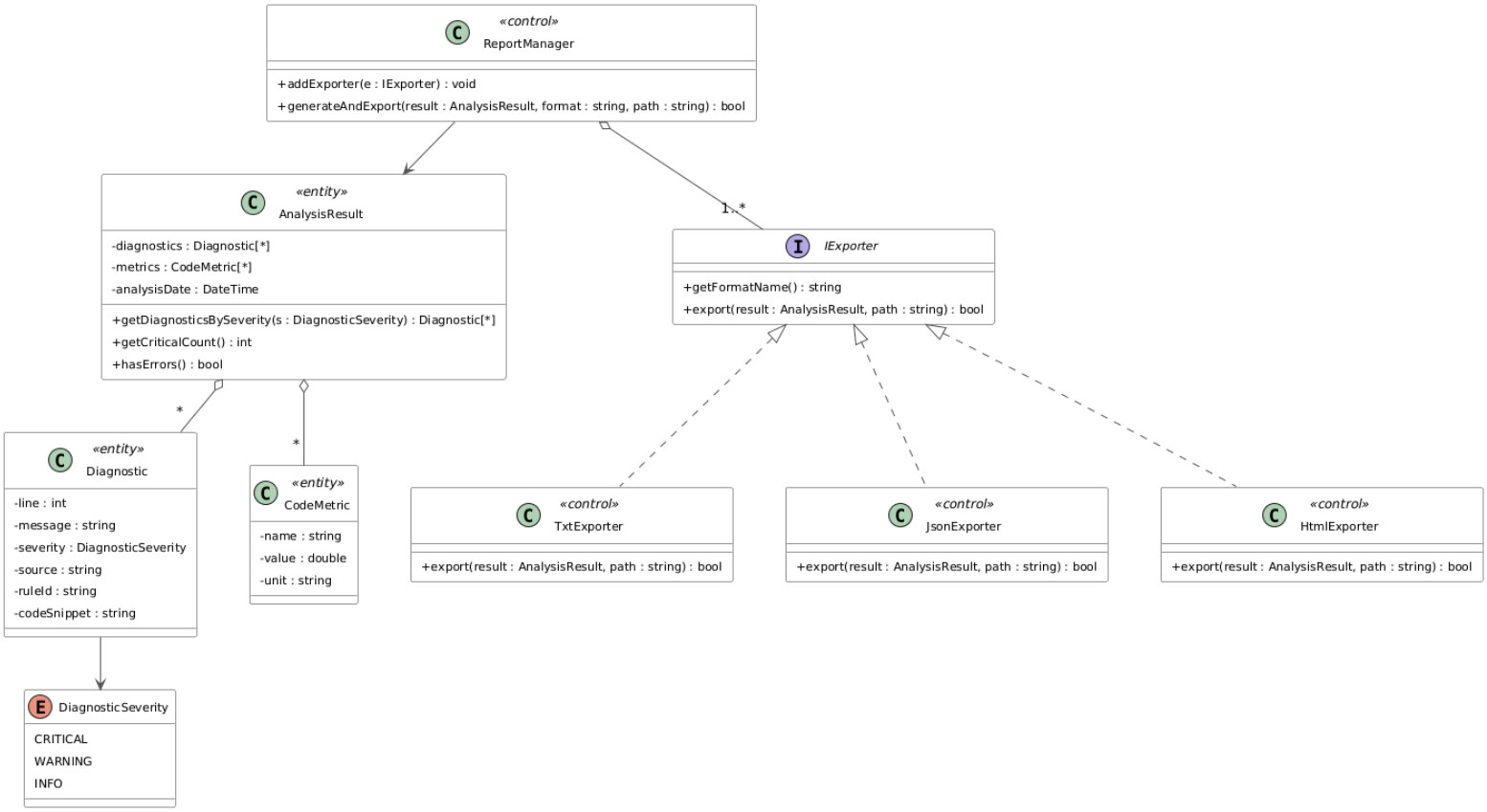
Ön İşleme, Lexer & Parser Modülü

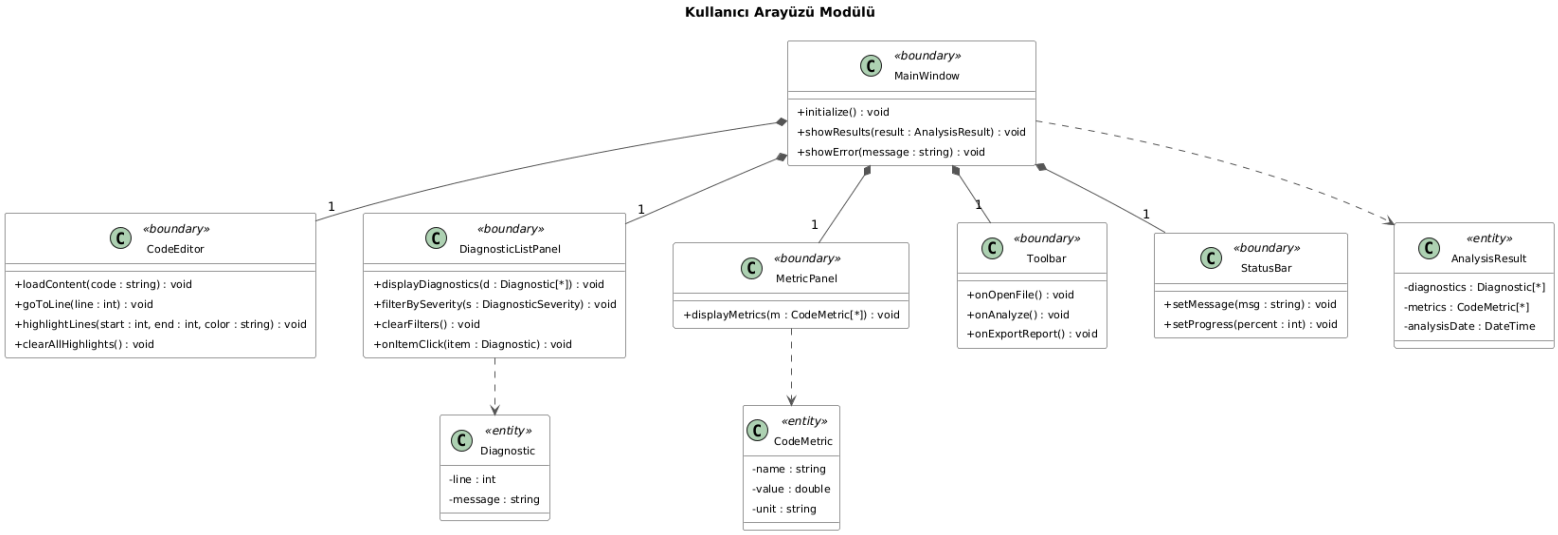


Analiz Motorları & Kural Motoru



Sonuçlar & Raporlama Modülü





Sınıf Açıklamaları

Uygulama Çekirdeği

1. Application «control»: Uygulamanın giriş noktasını oluşturan ve tüm sistemin yaşam döngüsünü yöneten üst düzey kontrol sınıfıdır. MainWindow ve AnalysisEngine nesnelerini oluşturarak sistemi başlatır, kapanış sırasında kaynakları serbest bırakır. Kullanıcı etkileşimlerini arayüzden alarak ilgili analiz operasyonlarına yönlendirir.
2. AnalysisEngine «control»: Tüm analiz sürecinin merkezi orkestratörüdür; UC-02 (Analiz Başlat) kullanım durumunun doğrudan karşılığıdır. Dosya yükleme, ön işleme, tokenization, AST oluşturma, syntax analizi, kural analizi, metrik hesaplama ve uyarı seviyesi sınıflandırma adımlarını doğru sırayla koordine eder.

Analiz tamamlandığında tüm çıktıları tek bir AnalysisResult nesnesinde birleştirir ve arayüze sunar.

Dosya Yönetimi

1. SourceFile «entity»: Kullanıcının sisteme yüklediği C kaynak kod dosyasını temsil eden varlık nesnesidir. Dosya yolu, dosya adı, metin içeriği, dosya boyutu ve yüklenme tarihi bilgilerini taşır. Analiz sürecinin tüm aşamalarında kaynak veriye erişim bu nesne üzerinden sağlanır.
2. FileLoader «control»: Diskten dosya okuma işlemini gerçekleştiren ve okunan içerikten bir SourceFile nesnesi oluşturan kontrol sınıfıdır. Dosya okuma öncesinde FileValidator aracılığıyla doğrulama yaparak yalnızca geçerli dosyaların sisteme alınmasını garanti eder. UC-01 (C Kodu Yükle) kullanım durumunun gerçekleştirimini sağlar.
3. FileValidator «control»: Yüklenen dosyanın analiz için uygunluğunu dört aşamalı doğrulama ile kontrol eden sınıftır: uzantı kontrolü (.c), boşluk kontrolü, okunabilirlik kontrolü ve binary dosya kontrolü. Sonucu bir ValidationResult nesnesi olarak döndürerek başarısızlık durumunda açıklayıcı hata mesajı sağlar (UC-03: Dosya Doğrula).
4. ValidationResult «entity»: Dosya doğrulama işleminin başarı durumunu ve olası hata mesajını taşıyan hafif veri nesnesidir. Doğrulama başarısız olduğunda sistemin analiz sürecini başlatmadan kullanıcıyı bilgilendirmesini sağlar.

Ön İşleme

1. Preprocessor «control»: Kaynak koddan tek satırlı (//) ve çok satırlı (/* */) yorumları ayıklayarak temizlenmiş kodu üreten ön işleme bileşenidir (UC-05: Yorum Satırlarını Ayıkla). Satır numarası bütünlüğünü koruyarak hata raporlamasının doğruluğunu garanti eder; string literal içindeki yorum benzeri dizileri bağlam duyarlı analiz ile ayrıştırarak yanlış pozitifleri önler.

Sözdizimsel Analiz

2. Lexer «control»: Temizlenmiş kaynak kodu sözlüksel analiz (lexical analysis) yoluyla anlamlı birimlere (token) ayıran bileşendir. Anahtar kelimeler, tanımlayıcılar, sayısal ve metin literalleri, operatörler ve ön işlemci direktifleri gibi token türlerini tanıyarak her birine satır ve sütun bilgisi ekler. Üretilen token dizisi, Parser sınıfının AST oluşturmasının temel girdisini oluşturur.

3. Token «entity»: Sözlüksel analiz sonucunda üretilen tek bir kod birimini temsil eden veri nesnesidir. Tokenın türünü (TokenType), metin değerini ve kaynak koddaki kesin konumunu (satır, sütun) tutar. Hem ayrıştırma sırasında hem de hata mesajlarında konum bilgisi sağlayıcı olarak kullanılır.
4. TokenType (enum): Bir tokenın kategorisini tanımlayan numaralandırmadır: KEYWORD, IDENTIFIER, INT_LITERAL, FLOAT_LITERAL, STRING_LITERAL, CHAR_LITERAL, OPERATOR, PUNCTUATION, PREPROCESSOR ve UNKNOWN. C diline özgü token çeşitliliğini doğru temsil ederek Parser'ın doğru sözdizimsel kararlar almasına olanak tanır.
5. Parser «control»: Token dizisini alarak sözdizimsel analiz yapan ve Soyut Sözdizim Ağacı (AST) oluşturan bileşendir (UC-04: Kod Ayrıştır). Fonksiyon tanımları, if/while/for yapıları, struct tanımları ve ifadeler gibi C dilinin yapısal bloklarını hiyerarşik ağaç düğümlerine dönüştürür. Ciddi syntax hatalarında bile kısmi AST üretir ve karşılaşılan sorunları Diagnostic listesine ekler.
6. ASTNode «entity»: Soyut Sözdizim Ağacı'ndaki tek bir düğümü temsil eden veri nesnesidir. Her düğüm; tipini (ASTNodeType), değerini, satır numarasını ve alt düğüm listesini tutar. Özyinelemeli yapısı sayesinde kaynak kodun tam yapısal hiyerarşisini modelleyerek syntax, kural ve metrik analizlerinin üzerinde gezineceği temel veri yapısını oluşturur.
7. ASTNodeType (enum): AST düğümlerinin türünü tanımlayan numaralandırmadır: PROGRAM, FUNCTION_DEF, VARIABLE_DECL, IF_STATEMENT, WHILE_LOOP, FOR_LOOP, RETURN_STMT, EXPRESSION, FUNCTION_CALL, STRUCT_DEF ve BLOCK. Bu ayrıntılı tür bilgisi, analiz motorlarının düğüm tipine göre özelleşmiş kontroller yapmasını sağlar.

Analiz Motorları

1. SyntaxAnalyzer «control»: AST ve token listesi üzerinde sözdizimi hatalarını tespit eden analiz bileşenidir (UC-06: Syntax Hatalarını Tespit Et). Eksik noktalı virgül, eşleşmeyen süslü/yuvarlak/köşeli parantezler ve geçersiz tip tanımlamaları gibi yaygın C syntax hatalarını kontrol eder. Her hata için satır bilgisi ve açıklayıcı mesaj içeren bir Diagnostic kaydı oluşturur.
2. RuleEngine «control»: Sisteme tanımlı tüm IRule nesnelerini yöneten ve AST üzerinde sırasıyla çalıştıran analiz koordinatörüdür (UC-07: Kural Analizi Yap). addRule() metodu ile kuralların çalışma zamanında dinamik olarak eklenmesini destekler. Tüm kuralların ürettiği bulguları toplayarak birleşik bir Diagnostic listesi döndürür.

3. IRule (Arayüz): Tüm kod kalitesi kurallarının uygulaması gereken sözleşmeyi tanımlayan saf sanal (pure virtual) arayüzdür. Her kuralın bir kimliği, adı ve AST üzerinde çalışan bir check() metodu olmalıdır. Strategy Pattern uygulaması sayesinde yeni kurallar mevcut kodu değiştirmeden sisteme eklenebilir (Açık/Kapalı Prensibi).
4. UnusedVariableRule «entity»: Kaynak kodda tanımlanmış ancak hiçbir yerde referans verilmemiş değişkenleri AST gezinerek tespit eden kural sınıfıdır. Tanımlanan ve kullanılan değişken kümelerini ayrı ayrı toplayarak farkını bulur. Gereksiz bellek kullanımını ve kod kirliliğini önlemeye yönelik bir kalite kuralıdır.
5. UninitializedVarRule «entity»: İlk değer atanmadan kullanılan değişkenleri tespit eden kural sınıfıdır. C dilinde başlatılmamış değişkenler belirsiz (undefined) davranışlara ve güvenlik açıklarına yol açabileceğinden bu kural kritik güvenilirlik denetimi sağlar.
6. MagicNumberRule «entity»: Kodda doğrudan kullanılan ve anlamı belli olmayan sayısal sabitleri (0 ve 1 gibi yaygın değerler hariç) tespit eden kural sınıfıdır. Sabitlerin isimlendirilerek tanımlanması kodun okunabilirliğini ve bakımını kolaylaştırır.
7. FunctionLengthRule «entity»: Yapılandırılabilir bir maksimum satır sayısını (varsayılan: 50) aşan fonksiyonları tespit eden kural sınıfıdır. Aşırı uzun fonksiyonlar test edilebilirliği ve okunabilirliği düşürdüğünden, bu kural fonksiyonların küçük ve odaklı tutulması yönünde uyarır.
8. GotoUsageRule «entity»: Kodda goto ifadesi kullanımını tespit ederek uyarı üreten kural sınıfıdır. goto C dilinde geçerli olmasına karşın yapısal programlama prensiplerine aykırıdır ve akış kontrolünü karmaşıklaştırır.
9. InfiniteLoopRule «entity»: Potansiyel sonsuz döngü yapılarını (örneğin koşulsuz while(1) veya güncellenmesi unutulan döngü değişkenleri) tespit eden kural sınıfıdır. Gereksinimlerdeki "sonsuz döngü potansiyeli" denetimine karşılık gelir.
10. MetricCalculator «control»: AST, ham kaynak kod ve temizlenmiş kod üzerinde nicel kod metriklerini hesaplayan bileşendir (UC-08: Metrik Hesapla). Toplam satır, kod satırı, boş satır, yorum satırı sayısı ile fonksiyon, struct, değişken sayısı ve ortalama fonksiyon uzunluğu gibi metrikleri hesaplar.

Sonuçlar ve Raporlama

1. DiagnosticSeverity (enum): Bulgulara atanan önem seviyelerini tanımlar: CRITICAL (kırmızı — derleme hatasına neden olan sorunlar), WARNING (sarı

— potansiyel sorunlar ve kötü pratikler), INFO (mavi — bilgilendirme ve stil önerileri). Arayüzde renk kodlaması ve filtreleme işlemlerinin temelini oluşturur.

2. Diagnostic «entity»: SyntaxError ve Finding kavramlarını tek bir sınıfta birleştiren birleşik bulgu nesnesidir. Satır numarası, mesaj, önem seviyesi, kaynağı (syntax/rule), kural kimliği ve ilgili kod parçası bilgilerini tek yapıda tutar. Bu birleştirme, arayüzde listeleme, filtreleme ve raporlama işlemlerini önemli ölçüde basitleştirir.
3. CodeMetric «entity»: Hesaplanan tek bir kod metriğini temsil eden veri nesnesidir. Adı, sayısal değeri ve birimini saklar. Metrik panelinde gösterim ve rapor oluşturma süreçlerinde yapılandırılmış veri kaynağı olarak kullanılır.
4. SeverityClassifier «control»: Her bulguya uygun önem seviyesi atan ve seviyeye göre renk kodu belirleyen sınıftır (UC-10: Uyarı Seviyesi Belirle). Sınıflandırma mantığını tek bir noktada toplayarak tutarlı seviye atanmasını garanti eder.
5. AnalysisResult «entity»: Bir analiz çalışmasının tüm çıktılarını tek nesnede birleştiren bütünleyici veri sınıfıdır. Tüm diagnostikleri, metrikleri, dosya bilgilerini ve analiz süresini tutar. getDiagnosticsBySeverity() ve getCriticalCount() gibi yardımcı sorgulama metotları ile arayüz ve rapor katmanlarında tekrarlanan filtreleme mantığını önler.
6. IExporter (Arayüz — Strategy Pattern): Rapor dışa aktarma formatlarının uygulaması gereken arayüzdür. Her format; format adını ve dışa aktarma mantığını kendi sınıfında tanımlar. Strategy Pattern sayesinde yeni format (PDF, CSV vb.) eklemek hiçbir mevcut sınıfı değiştirmeden yapılabilir.
7. TxtExporter, JsonExporter, HtmlExporter «control»: IExporter arayüzünü gerçekleştiren somut dışa aktarma sınıflarıdır. Her biri analiz sonuçlarını kendi formatına uygun biçimde dosyaya yazar. UC-12 (Analiz Sonucunu Kaydet) gereksinimindeki farklı format desteğinin doğrudan karşılığıdır.
8. ReportManager «control»: Dışa aktarma sürecini yöneten koordinatör sınıftır (UC-11: Rapor Oluştur). Birden fazla IExporter nesnesini bünyesinde barındırır ve kullanıcının seçtiği formata göre uygun exporter'ı çağırır.

Kullanıcı Arayüzü

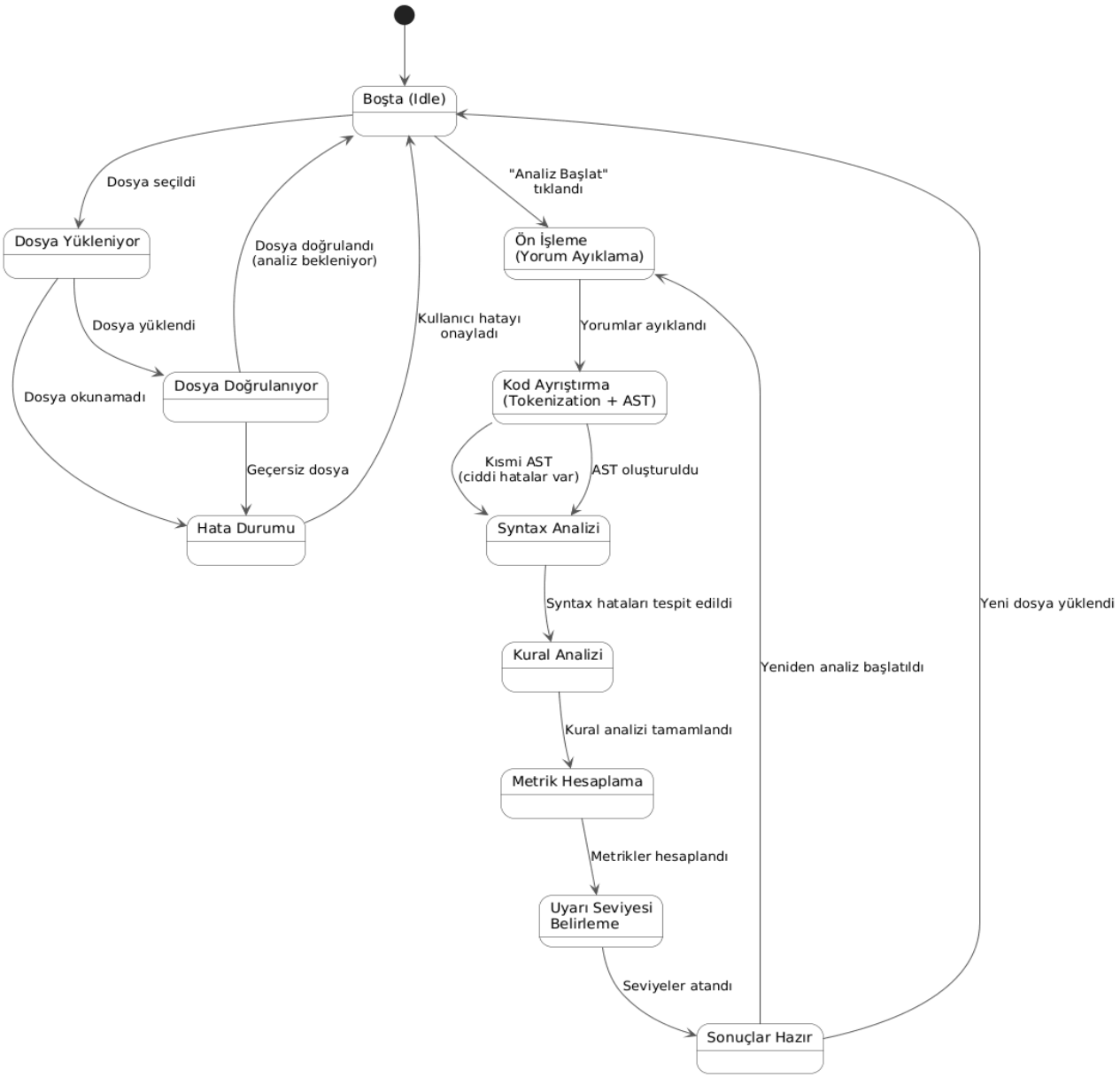
1. MainWindow «boundary»: Uygulamanın ana penceresini temsil eden birincil sınır nesnesidir. Kod editörü, bulgu listesi, metrik paneli, araç çubuğu ve durum çubuğu bileşenlerini bir arada barındırarak bütünleşik bir çalışma ortamı sunar. Analiz sonuçlarını ilgili panellere dağıtarak kullanıcının tüm bilgilere tek ekrandan erişmesini sağlar.

2. CodeEditor «boundary»: Kaynak kodun gösterildiği ve hata konumlarına navigasyon yapıldığı editör alanıdır. UC-13 (Kod Satırına Git) ve UC-15 (Kod Parçasını Vurgula) kullanım durumlarını destekler. Belirli satırların farklı renklerle vurgulanmasını sağlayarak kullanıcının hataları görsel olarak kavramasını kolaylaştırır.
3. DiagnosticListPanel «boundary»: Analiz bulgularını listeleyerek kullanıcıya sunan ve filtreleme imkânı sağlayan arayüz bileşenidir (UC-09: Sonuçları Görüntüle, UC-14: Hata Türüne Göre Filtrele). Önem seviyesine göre filtreleme yapabilir ve kullanıcının bir bulguya tıklaması halinde editör panelinde ilgili satıra yönlendirme olayını tetikler.
4. MetricPanel «boundary»: Hesaplanan kod metriklerini kullanıcıya özet biçimde gösteren arayüz bileşenidir. Kodun karmaşıklığı ve bakım durumu hakkında nicel bir genel değerlendirme sunarak kullanıcının kod kalitesini hızlıca kavramasını sağlar.
5. Toolbar «boundary»: Dosya açma, analiz başlatma ve rapor dışa aktarma gibi üst düzey işlemlerin düğmelerini barındıran araç çubuğu bileşenidir. Her düğme ilgili kullanım durumunu tetikleyerek sistemin ana işlevlerine doğrudan erişim sağlar.
6. StatusBar «boundary»: Kullanıcıya anlık durum mesajları ve analiz ilerleme bilgisi gösteren alt çubuk bileşenidir. Özellikle büyük dosyaların analizi sırasında kullanıcının sürecin hangi aşamada olduğunu görmesini sağlayarak kullanıcı deneyimini iyileştirir.

4.3 Durum Diyagramı

Bu bölümde sistemde durum değişimi gösteren önemli süreçler için durum diyagramları tanımlanmıştır.

4.3.1 Analiz Süreci Durum Diyagramı



Bu diyagram, analiz sürecinin baştan sona geçtiği tüm durumları ve durum geçişlerini modellemektedir.

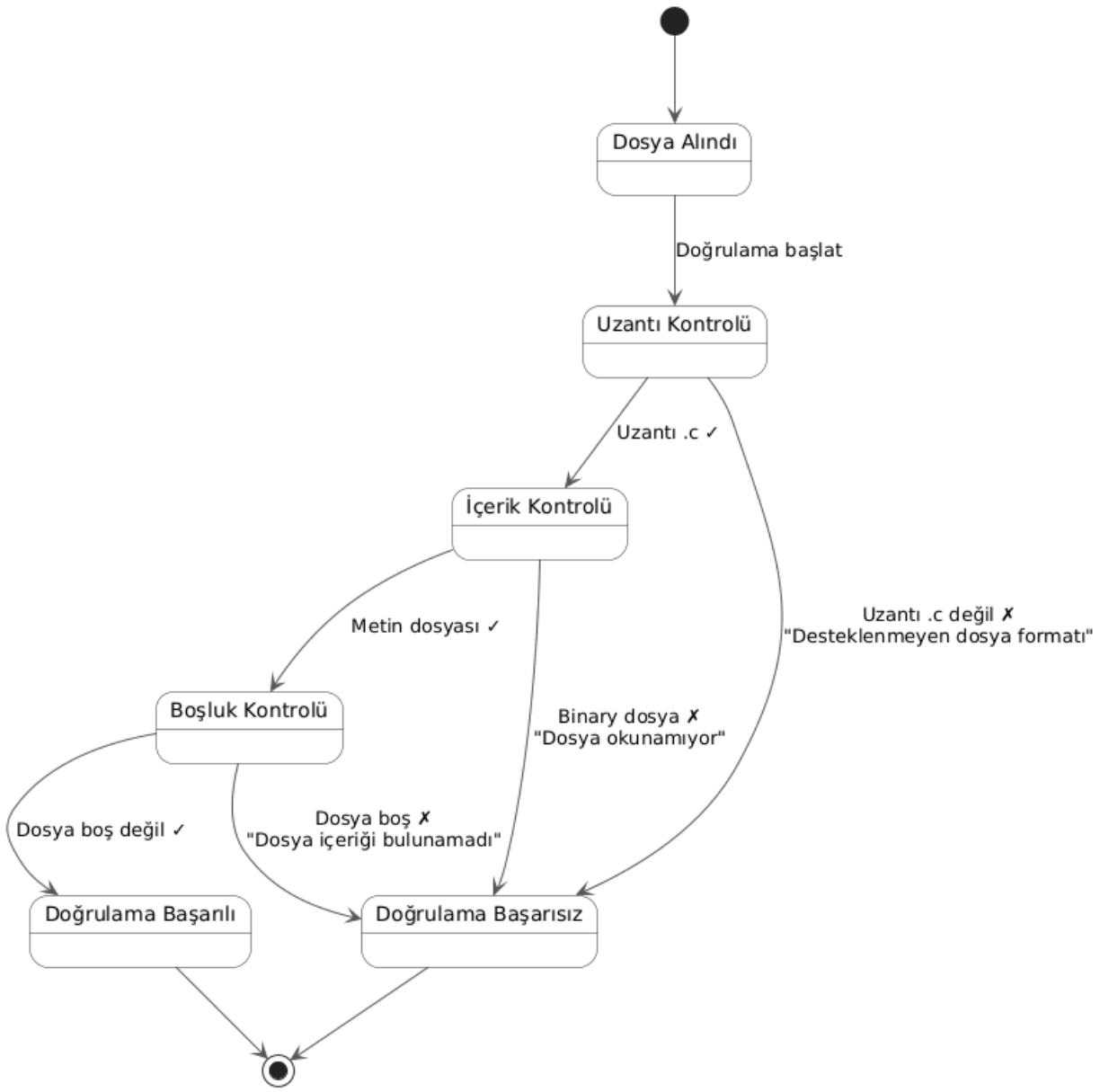
Sistem başlangıçta Boşta (Idle) durumundadır. Kullanıcı bir dosya seçtiğinde Dosya Yükleniyor durumuna geçer. Dosya başarıyla yüklenirse Dosya Doğrulanıyor durumuna, okunamazsa Hata Durumu'na geçiş yapılır.

Dosya doğrulandıktan sonra sistem tekrar Boşta durumuna döner ve kullanıcının "Analiz Başlat" komutunu bekler. Analiz başlatıldığında sırasıyla şu durumlardan geçer: Ön İşleme (Yorum Ayıklama) → Kod Ayrıştırma (Tokenization + AST) → Syntax Analizi → Kural Analizi → Metrik Hesaplama → Uyarı Seviyesi Belirleme → Sonuçlar Hazır.

Kod ayrıştırma sırasında ciddi syntax hataları oluşsa bile kısmi AST ile bir sonraki duruma geçilir. Sonuçlar hazır olduktan sonra kullanıcı yeni dosya yükleyebilir (Boşta'ya dönüş) veya yeniden analiz başlatabilir.

Herhangi bir adımda kritik hata oluşursa sistem Hata Durumu'na geçer; kullanıcı hatayı onayladığında Boşta durumuna döner.

4.3.2 Dosya Doğrulama Süreci Durum Diyagramı



Bu diyagram, dosya doğrulama sürecinin sıralı kontrol adımlarını ve olası sonuçlarını göstermektedir.

Dosya Alındı başlangıç durumundan sonra üç ardışık kontrol durumu bulunmaktadır:

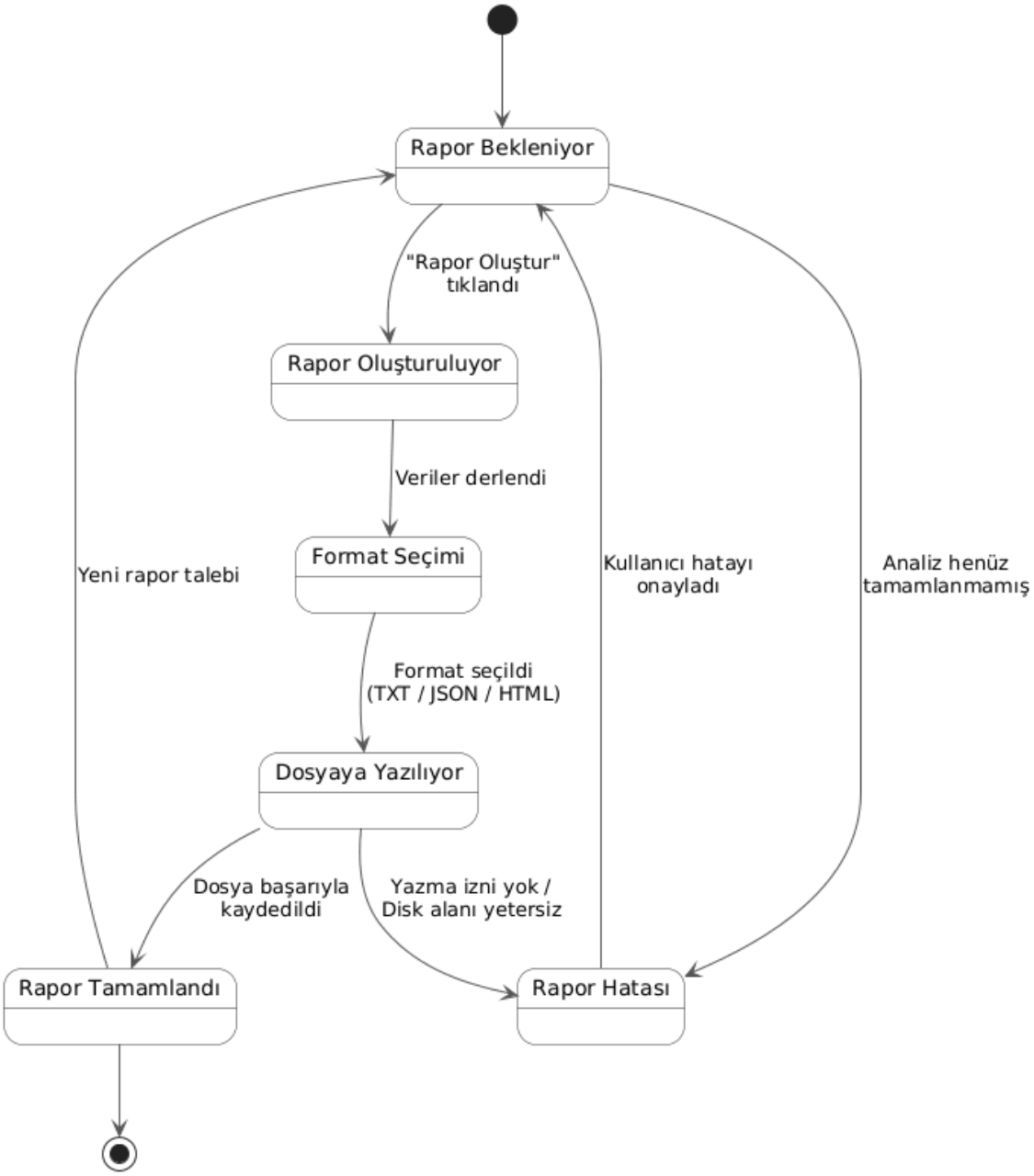
1. Uzantı Kontrolü: Dosyanın .c uzantısına sahip olup olmadığı kontrol edilir. Uygun değilse "Desteklenmeyen dosya formatı" hatası ile Doğrulama Başarısız durumuna geçilir.

2. İçerik Kontrolü: Dosyanın metin formatında (binary olmayan) olduğu doğrulanır. Binary dosya tespit edilirse "Dosya okunamıyor" hatası döner.

3. Boşluk Kontrolü: Dosyanın boş olup olmadığı kontrol edilir. Boşsa "Dosya içeriği bulunamadı" uyarısı ile başarısız duruma geçilir.

Üç kontrol de başarıyla geçilirse Doğrulama Başarılı son durumuna ulaşılır. Her kontrol noktası bir kapı görevi görür; herhangi birinde başarısızlık olması halinde süreç hemen sonlandırılır.

4.3.3 Rapor Oluşturma Süreci Durum Diyagramı



Bu diyagram, rapor oluşturma ve kaydetme sürecindeki durumları modellemektedir.

Başlangıç durumunda Rapor Bekleniyor ‘dur. Kullanıcı "Rapor Oluştur" butonuna tıkladığında, analiz tamamlanmışsa Rapor Oluşturuluyor durumuna geçilir; tamamlanmamışsa Rapor Hatası durumuna düşülür.

Veriler derlendikten sonra Format Seçimi durumuna geçilir (TXT, JSON veya HTML). Format belirlendikten sonra Dosyaya Yazılıyor durumuna geçilir. Dosya başarıyla kaydedilirse Rapor Tamamlandı son durumuna ulaşılır. Yazma izni yoksa veya disk alanı yetersizse Rapor Hatası durumuna geçiş yapılır.

Hem tamamlanma hem de hata durumlarından Rapor Bekleniyor durumuna dönüş mümkündür, böylece kullanıcı yeni bir rapor talebi oluşturabilir.

4.4 Aktivite Diyagramı

Aktivite diyagramları sistemin iş akışlarını göstermektedir.

4.4.1 Genel Analiz Süreci Aktivite Diyagramı

Kullanıcı .c dosyasını seçer
Dosya sisteme yüklenir
Dosya doğrulaması yapılır
Dosya geçerli mi?
Evet
Hayır
Kullanıcı "Analiz Başlat" butonuna tıklar
Hata mesajı göster
Analiz sürecini iptal et
Yorum satırları ayıklanır (tek satır & çok satır)
Kod tokenlara ayrılır (Lexical Analysis)
AST oluşturulur (Abstract Syntax Tree)
AST başarıyla oluşturuldu mu?
Evet
Hayır
Tam AST
Kısmi AST (hatalar kaydedildi)
Syntax hatalarını tespit et
Kural analizi yap
Metrik hesaplama yap
Uyarı seviyeleri atanır (Kritik / Uyarı / Bilgi)
Sonuçlar arayüzde gösterilir
Rapor isteniyor mu?
Hayır
Evet
Rapor oluştur ve kaydet

Bu diyagram, sistemin uçtan uca analiz iş akışını göstermektedir.

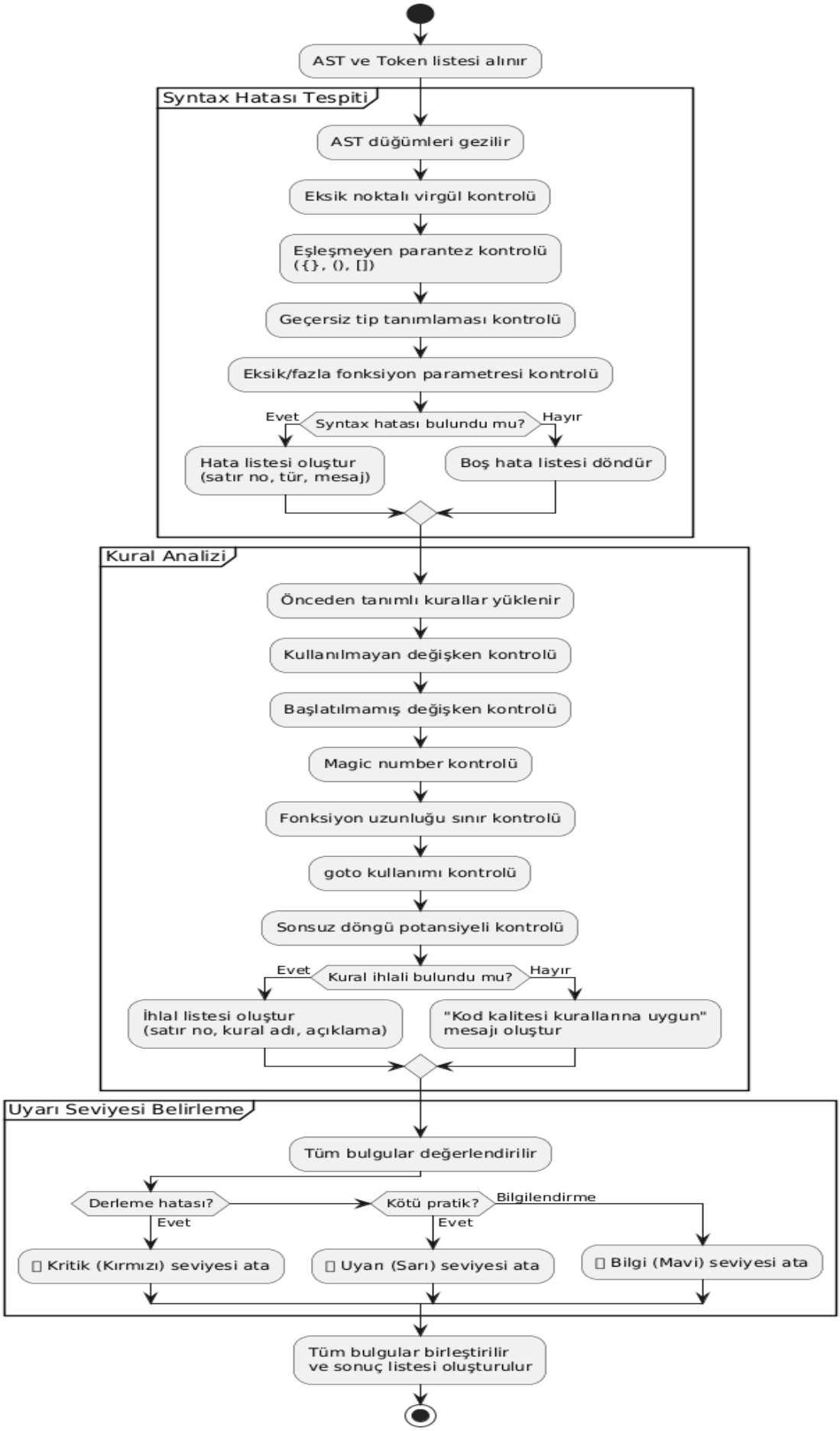
Süreç, kullanıcının .c dosyasını seçmesiyle başlar. Dosya yüklenir ve doğrulama yapılır. Doğrulama başarısız olursa hata mesajı gösterilip süreç sonlandırılır.

Dosya geçerliyse kullanıcı "Analiz Başlat" butonuna tıklar. Sırasıyla yorum ayıklama, tokenization ve AST oluşturma adımları gerçekleştirilir. AST oluşturma sonucunda tam veya kısmi (hatalar kaydedilerek) ağaç elde edilir.

Ardından diyagramda bir fork (paralel dallanma) yapısı kullanılmıştır: syntax hata tespiti, kural analizi ve metrik hesaplama eş zamanlı olarak yürütülür. Bu paralel yapı, analiz performansını artırmak amacıyla tasarlanmıştır.

Paralel işlemler tamamlandıktan sonra uyarı seviyeleri (Kritik/Uyarı/Bilgi) atanır ve sonuçlar arayüzde gösterilir. Kullanıcı isterse rapor oluşturma ve kaydetme adımına geçebilir.

4.4.2 Hata Analizi Süreci Aktivite Diyagramı


AST ve Token listesi alınır
Syntax Hatası Tespiti
AST düğümleri gezilir
Eksik noktalı virgül kontrolü
Eşleşmeyen parantez kontrolü
({}, (), [])
Geçersiz tip tanımlaması kontrolü
Eksik/fazla fonksiyon parametresi kontrolü
Syntax hatası bulundu mu?
Evet
Hayır
Hata listesi oluştur
(satır no, tür, mesaj)
Boş hata listesi döndür
Kural Analizi
Önceden tanımlı kurallar yüklenir
Kullanılmayan değişken kontrolü
Başlatılmamış değişken kontrolü
Magic number kontrolü
Fonksiyon uzunluğu sınır kontrolü
goto kullanımı kontrolü
Sonsuz döngü potansiyeli kontrolü
Kural ihlali bulundu mu?
Evet
Hayır
İhlal listesi oluştur
(satır no, kural adı, açıklama)
"Kod kalitesi kurallarına uygun"
mesajı oluştur
Uyarı Seviyesi Belirleme
Tüm bulgular değerlendirilir
Derleme hatası?
Kötü pratik?
Bilgilendirme
Evet
Evet
Kritik (Kırmızı) seviyesi ata
Uyarı (Sarı) seviyesi ata
Bilgi (Mavi) seviyesi ata
Tüm bulgular birleştirilir
ve sonuç listesi oluşturulur

Bu diyagram, hata tespit ve değerlendirme sürecinin detaylı iş akışını göstermektedir. Üç ana bölümden (partition) oluşur:

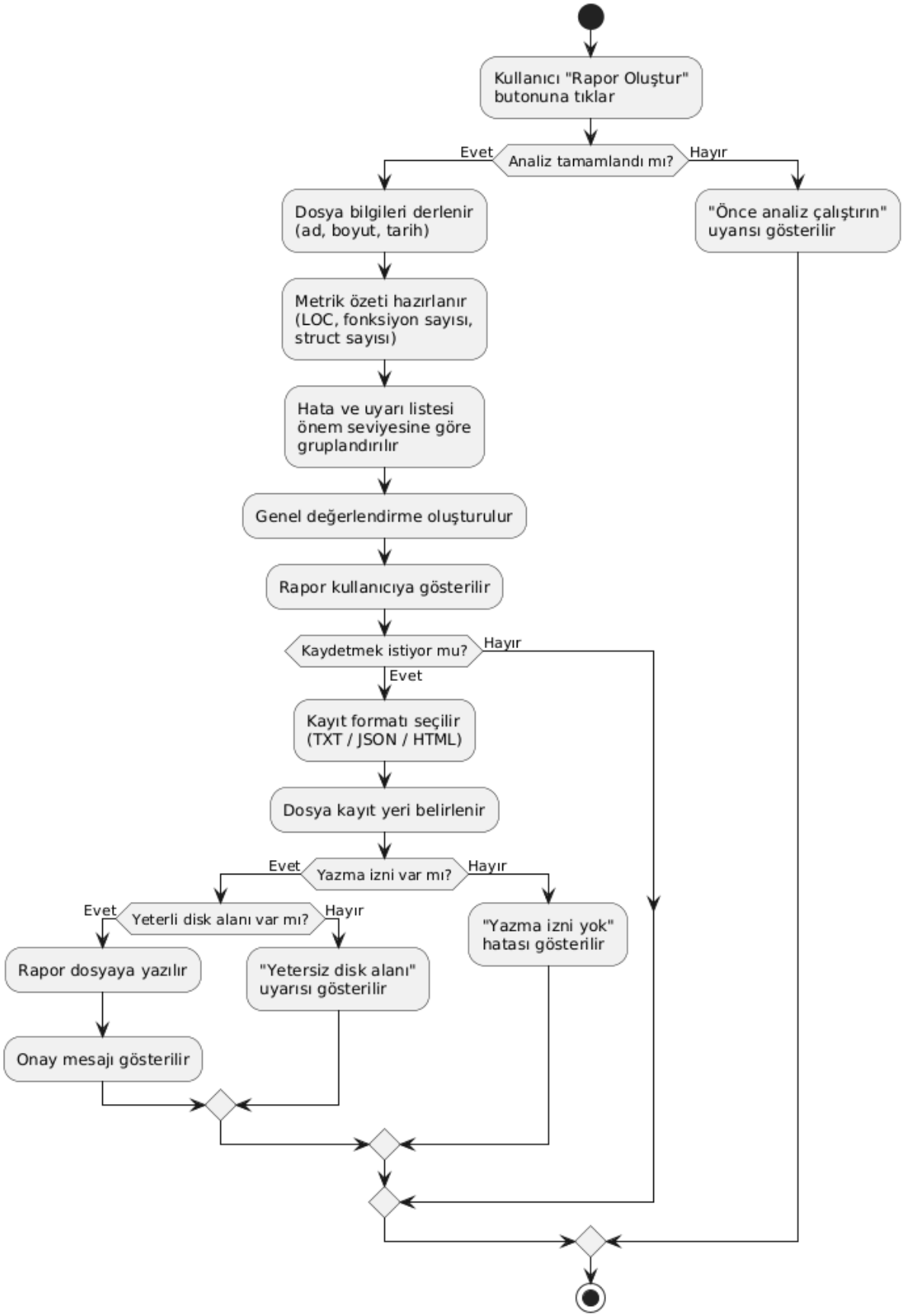
Syntax Hatası Tespiti bölümünde AST ve token listesi üzerinden sırasıyla şu kontroller yapılır: eksik noktalı virgül, eşleşmeyen parantez ({}, (), []), geçersiz tip tanımlaması ve eksik/fazla fonksiyon parametresi. Hata bulunursa satır numarası, tür ve mesaj bilgileriyle hata listesi oluşturulur; bulunmazsa boş liste döner.

Kural Analizi bölümünde önceden tanımlı kurallar yüklenir ve sırasıyla şu kontroller gerçekleştirilir: kullanılmayan değişken, başlatılmamış değişken, magic number, fonksiyon uzunluğu sınırı, goto kullanımı ve sonsuz döngü potansiyeli. İhlal bulunursa ihlal listesi oluşturulur; bulunmazsa "Kod kalitesi kurallarına uygun" mesajı üretilir.

Uyarı Seviyesi Belirleme bölümünde her bulgu değerlendirilir: derleme hatasına neden olacak sorunlara Kritik (Kırmızı), potansiyel sorun ve kötü pratiklere Uyarı (Sarı), bilgilendirme amaçlı önerilere Bilgi (Mavi) seviyesi atanır.

Son adımda tüm bulgular birleştirilir ve tek bir sonuç listesi oluşturulur.

4.4.3 Rapor Oluşturma Süreci Aktivite Diyagramı


Kullanıcı "Rapor Oluştur" butonuna tıklar
Analiz tamamlandı mı?
Evet
Hayır
Dosya bilgileri derlenir (ad, boyut, tarih)
"Önce analiz çalıştırın" uyarısı gösterilir
Metrik özeti hazırlanır (LOC, fonksiyon sayısı, struct sayısı)
Hata ve uyarı listesi önem seviyesine göre gruplandırılır
Genel değerlendirme oluşturulur
Rapor kullanıcıya gösterilir
Kaydetmek istiyor mu?
Hayır
Evet
Kayıt formatı seçilir (TXT / JSON / HTML)
Dosya kayıt yeri belirlenir
Yazma izni var mı?
Evet
Hayır
Yeterli disk alanı var mı?
Evet
Hayır
"Yazma izni yok" hatası gösterilir
Rapor dosyaya yazılır
"Yetersiz disk alanı" uyarısı gösterilir
Onay mesajı gösterilir

Bu diyagram, rapor oluşturma ve kaydetme sürecinin tüm karar noktalarını ve alternatif akışlarını göstermektedir.

Süreç kullanıcının "Rapor Oluştur" butonuna tıklamasıyla başlar. İlk karar noktasında analizin tamamlanıp tamamlanmadığı kontrol edilir; tamamlanmamışsa "Önce analiz çalıştırın" uyarısı gösterilir.

Analiz tamamlanmışsa dört adım sırasıyla gerçekleştirilir:

1. Dosya bilgileri (ad, boyut, tarih) derlenir.
2. Metrik özeti (LOC, fonksiyon sayısı, struct sayısı) hazırlanır.
3. Hata ve uyarı listesi önem seviyesine göre gruplandırılır.
4. Genel değerlendirme oluşturulur.

Rapor kullanıcıya gösterildikten sonra kaydetme seçeneği sunulur. Kullanıcı kaydetmek isterse format seçimi (TXT/JSON/HTML) yapılır ve dosya kayıt yeri belirlenir. Ardından iki güvenlik kontrolü uygulanır: yazma izni kontrolü ve disk alanı kontrolü. Her iki kontrol de başarılıysa rapor dosyaya yazılır ve onay mesajı gösterilir; aksi halde ilgili hata mesajları kullanıcıya sunulur.

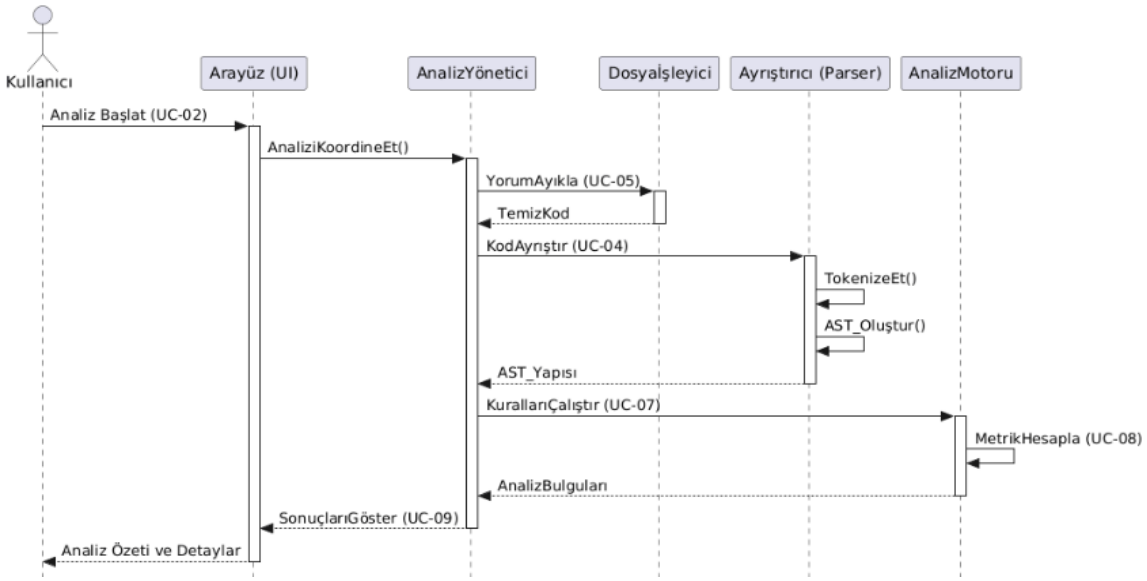
4.5 Sıra (Sequence) Diyagramı

Sıra diyagramları, sistemdeki nesnelerin ve aktörlerin belirli bir kullanım durumu (Use Case) çerçevesinde zaman dizini boyunca gerçekleştirdikleri etkileşimleri modeller. Bu bölümde, "Statik Kod Analizi ve Hata Tespit Sistemi"nin modüler yapısını ve nesne yönelimli prensiplerle kurgulanan iş akışlarını temsil eden en kritik dört senaryo analiz edilmiştir. Diyagramlarda kullanılan temel bileşenler; kullanıcı arayüzü (UI), analiz yönetim merkezi, dosya işleme birimi, ayrıştırıcı (parser) ve analiz motoru nesneleridir.

4.5.1 Senaryo 1: Standart Analiz ve Başarılı Akış (Happy Path)

Bu senaryo, sistemin temel varlık sebebini ve tüm modüllerin uyum içinde çalıştığı ideal iş akışını modeller. Süreç, kullanıcının geçerli bir .c dosyasını sisteme yüklemesiyle başlar.

- Başlatma: Kullanıcı arayüz üzerinden analiz komutunu verir (UC-02).
- Ön İşleme: Analiz yöneticisi, ilk olarak dosya işleyiciyi tetikleyerek tek ve çok satırlı yorumları ayıklar (UC-05), bu sırada satır numarası bütünlüğünü korur.
- Ayrıştırma (Parsing): Kod, sözcük dizimsel analize (Tokenization) tabi tutulur ve kodun hiyerarşik yapısını temsil eden Soyut Sözdizimi Ağacı (AST) oluşturulur (UC-04).
- Denetim ve Hesaplama: Oluşturulan AST üzerinden eş zamanlı olarak kod kalitesi kuralları çalıştırılır (UC-07) ve kod metrikleri (LOC, fonksiyon sayısı vb.) hesaplanır (UC-08).
- Sonuçlandırma: Elde edilen tüm bulgular analiz yöneticisi tarafından toplanarak kullanıcı arayüzüne "başarılı" statüsüyle iletilir ve listelenir (UC-09).
-

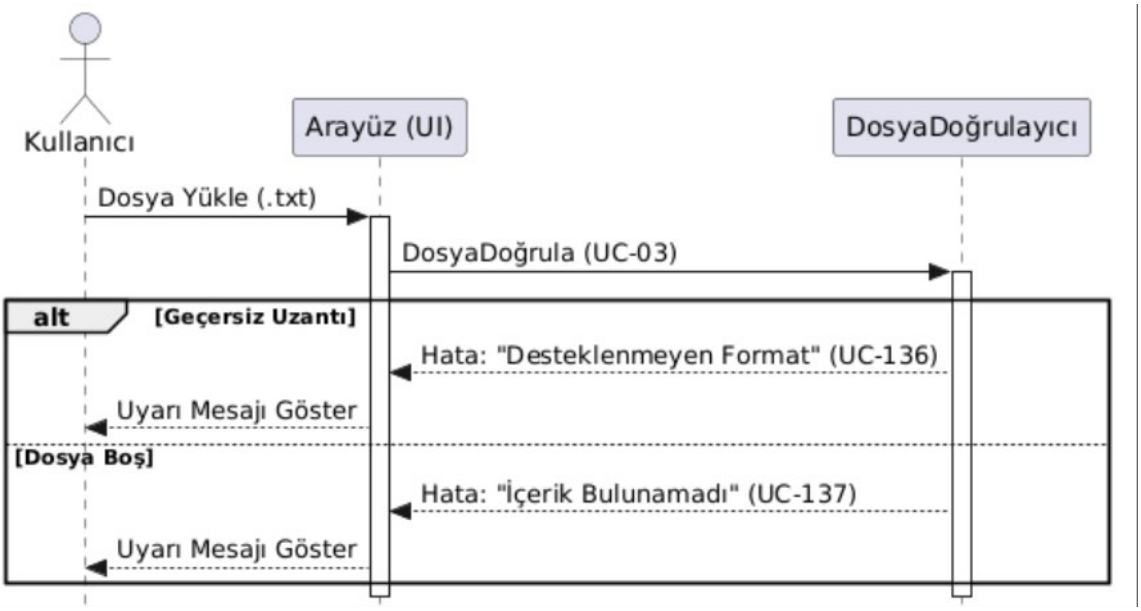


[Görsel Alanı: Şekil 4.5.1 - Başarılı Analiz Akışı Sıra Diyagramı]

4.5.2 Senaryo 2: Geçersiz Giriş ve Hata Yönetimi

Bu senaryo, sistemin işlevsel olmayan gereksinimlerinden olan "güvenilirlik" prensibini test eder. Kullanıcının hatalı bir dosya yüklemesi durumunda sistemin nasıl bir koruma kalkanı oluşturduğu modellenir.

- Doğrulama: Kullanıcı dosya seçtiği anda sistem otomatik olarak dosya doğrulama sürecini başlatır (UC-03).
- Format Kontrolü: Sistem dosya uzantısının .c olup olmadığını ve dosyanın boş olup olmadığını kontrol eder.
- Hata Yakalama: Eğer dosya formatı uygun değilse veya dosya okunamaz/bozuksa, süreç analiz aşamasına geçmeden kesilir.
- Geri Bildirim: Hata yakalayıcı nesne, kullanıcıya arayüz üzerinden anlamlı bir uyarı mesajı (örn: "Desteklenmeyen dosya formatı") dönerek etkileşimi sonlandırır.

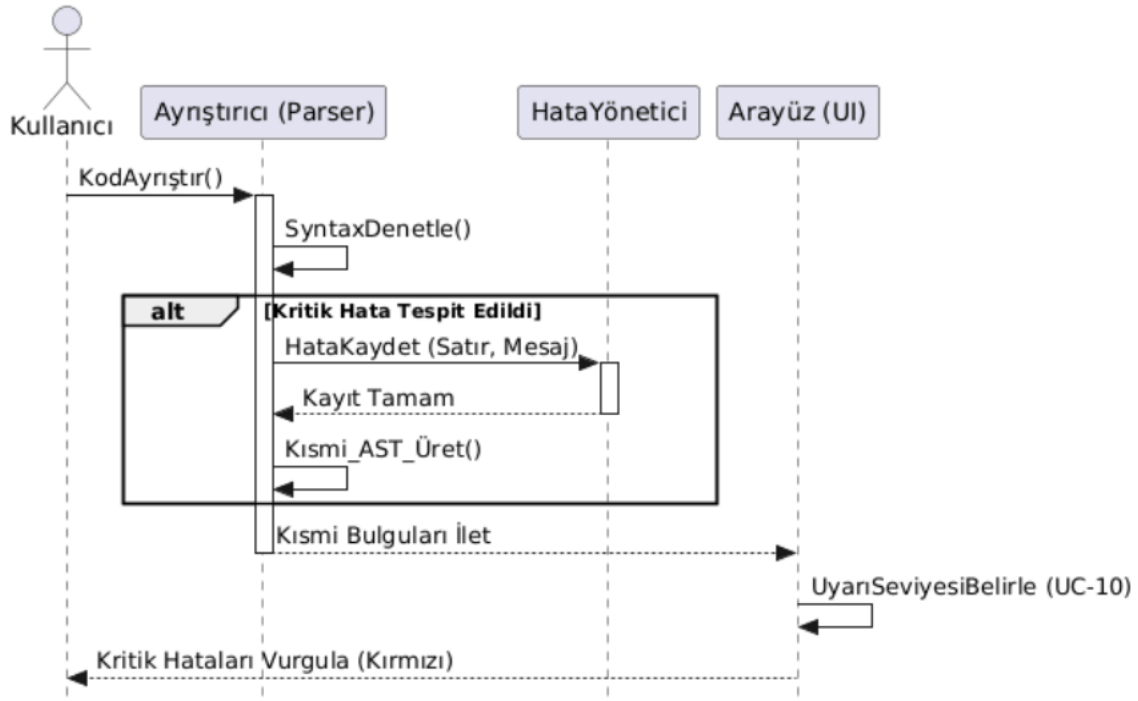


[Görsel Alanı: Şekil 4.5.2 - Dosya Doğrulama ve Hata Yakalama Sıra Diyagramı]

4.5.3 Senaryo 3: Sözdizimi (Syntax) Hatası ve Görsel Vurgulama

Analiz edilen kodda yapısal hatalar bulunması durumunda, sistemin bu hataları kullanıcıya nasıl navigasyonel olarak sunduğunu temsil eden senaryodur.

- Tespit: Ayrıştırıcı (Parser), AST oluşturma sırasında eksik noktalı virgül veya eşleşmeyen parantez gibi kritik hatalarla karşılaşır (UC-06).
- Kategorizasyon: Sistem, tespit edilen her hata için bir önem seviyesi ve renk kodu atar (UC-10). Kritik hatalar kırmızı olarak işaretlenir.
- Navigasyon: Kullanıcı sonuç listesindeki bir hataya tıkladığında, arayüz otomatik olarak editördeki ilgili satıra kaydırma yapar (UC-13).
- Vurgulama: Sistem, hatanın bulunduğu kod bloğunu görsel olarak renklendirerek kullanıcının hata tespiti sürecini kolaylaştırır (UC-15).

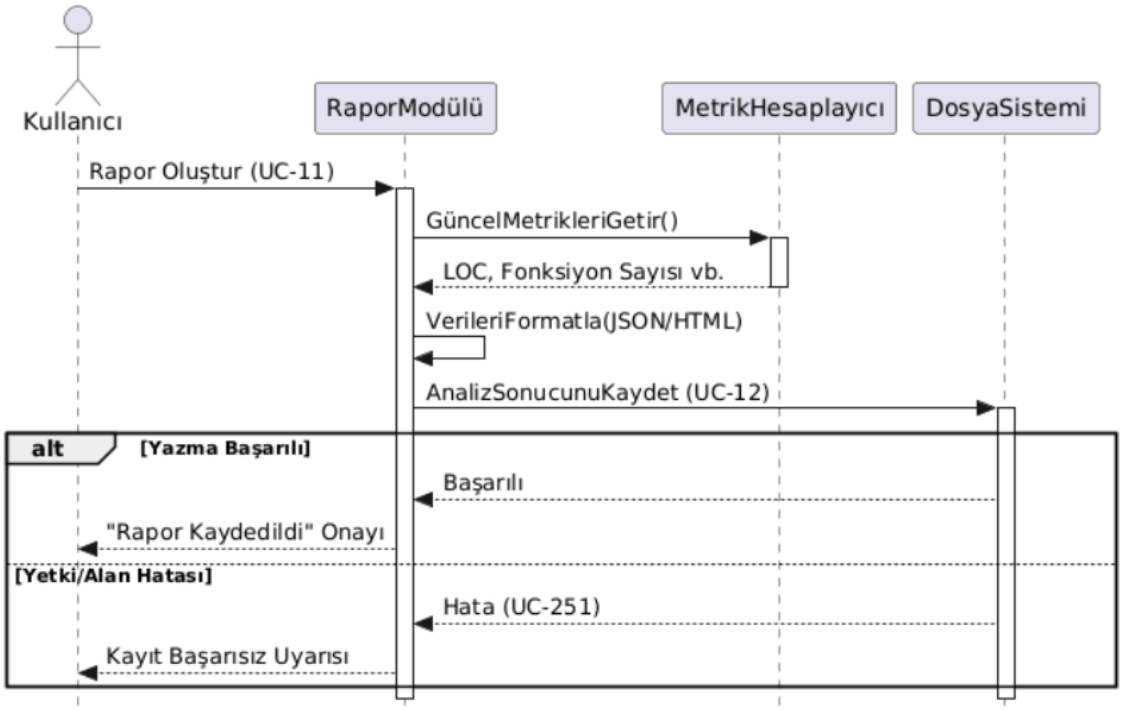


[Görsel Alanı: Şekil 4.5.3 - Syntax Hatası Tespiti ve Navigasyon Sıra Diyagramı]

4.5.4 Senaryo 4: Analiz Verilerinin Raporlanması ve Kaydı

Bu senaryo, analizi tamamlanmış verilerin bir çıktıya dönüştürülmesi ve kalıcı depolama birimine yazılması sürecini kapsar.

- Derleme: Kullanıcı "Rapor Oluştur" butonuna tıkladığında (UC-11), sistem mevcut analiz sonuçlarını ve hesaplanan metrikleri (LOC, struct sayısı vb.) bir araya getirir.
- Yapılandırma: Rapor modülü, ham verileri kullanıcı tarafından seçilen (HTML, JSON veya TXT) formata dönüştürür.
- Kayıt İşlemi: Hazırlanan rapor verisi, dosya sistemi nesnesi üzerinden yerel diske yazılır (UC-12).
- Onay: İşlem sonunda kullanıcıya "Rapor başarıyla kaydedildi" veya bir sorun oluştuysa "Yazma izni yok" gibi durum bildirimleri yapılır.



[Görsel Alanı: Şekil 4.5.4 - Rapor Oluşturma ve Dışa Aktarma Sıra Diyagramı]

4.6 Kullanıcı Arayüzü

Kullanıcı arayüzü sistemin kullanıcı ile etkileşim kurduğu katmandır.

Dosya Analiz Rapor Yardım

Kod Editörü | Analiz Sonuçları | Metrikler

C Kaynak Kodu Editörü

```
1 #include <stdio.h>
2
3 int main()
4     int x;
5     printf("%d", x)    // <-- eksik ;
6     int y = 42;        // <-- magic number
7     int z = 10;
8     // z kullanılmıyor
9     return 0;
10
```

Dosya Aç | Analiz Başlat | Rapor Oluştur | Kaydet

Analiz Sonuçları

Filtre: Tümü ▼ (Kritik, Uyarı, Bilgi) | Filtrele

Seviye	Satır	Tür	Açıklama
Kritik	5	Syntax Hatası	Eksik noktalı virgül (;)
Uyarı	6	Magic Number	Sabit yerine sayısal değer (42) kullanımı
Uyarı	4	Başlatılmamış Değ.	"x" değişkeni başlatılmadan kullanılıyor
Uyarı	7	Kullanılmayan Değ.	"z" değişkeni tanımlanmış ama kullanılmamış

Metrik Özeti:

Metrik	Değer
Toplam Satır (LOC)	10
Fonksiyon Sayısı	1
Struct Sayısı	0
Boş Satır Sayısı	1
Değişken Sayısı	3

✔ Analiz tamamlandı Toplam: 4 bulgu (1 Kritik, 3 Uyarı, 0 Bilgi) Süre: 0.12s

5. Referanslar

- B. Chess and J. West, Secure Programming with Static Analysis. Upper Saddle River, NJ: Addison-Wesley, 2007
- F. Nielson, H. R. Nielson, and C. Hankin, Principles of Program Analysis. Berlin, Germany: Springer-Verlag, 1999.
- P. Emanuelsson and U. Nilsson, "A Comparative Study of Industrial Static Analysis Tools," Electronic Notes in Theoretical Computer Science, vol. 217, pp. 5–21, 2008.
- A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed. Boston, MA: Pearson, 2007.
- A. W. Appel, Modern Compiler Implementation in C. Cambridge, UK: Cambridge University Press, 1998.